

Minería de Datos Aplicada a México: Sistema de Transporte Colectivo

Arlet Pinacho Báez (320287828)

Isaac Giovani Escobar González (321336400)

Kevin Jonathan Garduño Escobar (321070629)

Seldon Sautto Ramírez (321084163)

2026-06-01

Table of contents

1	Proyecto Final - Minería de Datos Aplicada a México	5
2	Afluencia en el STC de la CDMX	6
2.1	Descripción del dataset	7
2.1.1	Diccionario de Datos	8
2.1.2	Consideraciones sobre el tipo de pago	10
2.1.3	Limitaciones del conjunto de datos	11
3	Limpieza del dataset	12
3.1	Valores registrados	12
3.2	Limpieza de acentos	13
3.3	Integridad de fechas	15
3.4	Datos duplicados	15
4	Preparación del dataset	16
4.1	Modelo para clasificación	18
4.1.1	Variable objetivo	18
4.2	Modelo para clustering	21
5	EDA — Análisis Exploratorio de Datos	22
5.1	Carga del Dataset	22
5.2	Distribución de la Afluencia Total (2021–2026)	23
5.3	Interpretación Inicial	24
5.4	Detección de Valores Atípicos	25
5.4.1	Interpretación	25
5.5	Afluencia Promedio por Línea	26
5.5.1	Visualización de la Afluencia Media por Línea	26
5.5.2	Top 5 Líneas con Mayor Afluencia Promedio	27
5.6	Interpretación	28
5.7	Evolución Temporal de la Afluencia (2021–2026)	28
5.7.1	Visualización de la Evolución Anual	29
5.8	Interpretación Temporal	30
5.9	Comparación de Afluencia: Antes y Después del Fin del Boleto Físico	30
5.9.1	Visualización Comparativa	31
5.10	Interpretación	32

5.11	Evolución de la Movilidad por Mes (2021–2026)	33
5.11.1	Visualización Mensual	34
5.12	Interpretación Mensual	35
5.13	Evolución Temporal: Línea 2 vs Línea 3	35
5.13.1	Visualización Comparativa	36
5.14	Interpretación Comparativa	37
5.15	Relaciones entre Componentes de Afluencia	38
5.15.1	Interpretación	39
6	Conclusión General del Análisis Exploratorio	41
7	Modelo de clasificación	43
7.1	Configuración del entorno y librerías	43
7.2	Carga de datos e información del dataset	45
7.3	Definición de la variable objetivo y predictoras	47
7.3.1	Variable objetivo	47
7.3.2	Variables predictoras incluidas	49
7.3.3	Variables excluidas y justificación	50
7.4	Diseño e implementación con POO: Patrón Strategy y Pipeline	54
7.5	División del conjunto de datos	55
7.6	Modelo Base (Baseline)	57
7.7	Entrenamiento y Ajuste de Hiperparámetros	57
7.8	Análisis de los Hiperparámetros	60
7.9	Evaluación de Rendimiento y Métricas	62
7.9.1	Justificación de Métricas Seleccionadas	62
7.9.2	Cálculo y Presentación de Métricas	63
7.9.3	Análisis e interpretación de resultados	65
7.10	Visualización: Matriz de Confusión y Curva ROC	67
7.10.1	Matriz de Confusión	67
7.10.2	Curvas ROC — One-vs-Rest (OvR)	68
7.11	Análisis de Importancia de Variables	70
7.12	Persistencia del Modelo y Demostración de Reutilización	72
7.12.1	Guardado del modelo	72
7.12.2	Demostración de reutilización (celda independiente)	73
8	Clustering — Agrupamiento de Estaciones	76
8.1	1. Selección y Justificación de Variables	76
8.1.1	Justificación	76
8.2	2. Carga de Datos y configuración del Entorno	77
8.3	3. Preprocesamiento	78
8.3.1	Justificación	79
8.4	4. Selección del Algoritmo y Justificación	79
8.4.1	Justificación del uso de K-Means	79

8.5	5. Elección del Número de Grupos	80
8.5.1	5.1 Método del Codo (Inercia)	80
8.5.2	Selección Final de (k)	82
8.6	6. Aplicación del Modelo K-Means	82
8.6.1	6.1 Interpretación Inicial de los Clusters	83
8.7	7. Perfilado de los Clusters	84
8.7.1	7.1 Centroides en Escala Original	84
8.7.2	7.2 Perfiles Estadísticos por Cluster	84
8.7.3	7.3 Visualización de Perfiles	85
8.8	8 Interpretación de Perfiles	86
8.8.1	Cluster 0 — Alta Demanda Estructural	86
8.8.2	Cluster 1 — Demanda Intermedia	86
8.8.3	Cluster 2 — Baja Intensidad Estructural	86
8.9	9. Asignación de Etiquetas Interpretativas	87
8.10	10. Visualización de Clusters	88
8.10.1	10.1 Visualización de Clusters con PCA	88
8.10.2	10.2 Distribución de Estaciones por Cluster	89
8.10.3	10.3 Distribución de Clusters por Línea	90
8.11	11. Persistencia del Modelo	90
8.12	12 Interpretación Final del Clustering	91
9	Arquitectura y diseño	92
9.1	1. Arquitectura orientada a objetos	92
9.1.1	1.1 Visión General del Módulo <code>src/</code>	92
9.1.2	1.2 Clases y Responsabilidades	92
9.1.3	1.3 Módulos de Preprocesamiento	93
9.1.4	1.4 Script de demostración	93
9.2	2. Patrones de diseño aplicados	94
9.2.1	2.1 Patrón Strategy	94
9.2.2	2.2 Patrón Pipeline	94
9.3	3. Diagrama de clases	96
9.4	4. Persistencia de modelos	97
10	Conclusiones	98
10.1	1. Hallazgos principales	98
10.2	2. Limitaciones del análisis	99
10.3	3. Trabajo Futuro	99
	Referencias	100

1 Proyecto Final - Minería de Datos Aplicada a México

En este documento se describe la realización paso a paso del proyecto final para la materia de Almacenes y Minería de Datos. Dicho proyecto tiene como objetivo principal, no solo aplicar los conocimientos clave adquiridos a lo largo del curso por nuestro equipo, sino además permitir el desarrollo de una solución completa de minería de datos orientada a un problema real de México.

Para ello, se exploran los siguientes puntos:

- Problemática a tratar
- Introducción al conjunto de datos
- Limpieza y preparación del conjunto de datos
- EDA
- Modelo supervisado
 - Implementación
 - Métricas
 - Análisis de resultados
- Modelo de Clustering
 - Implementación
 - Visualización e interpretación de perfiles.
- Arquitectura y diseño
- Reutilización del modelo
- Conclusiones
 - Hallazgos
 - Limitaciones
 - Trabajo futuro
- Referencias

2 Afluencia en el STC de la CDMX

El *Sistema de Transporte Colectivo* del Valle de México es un organismo descentralizado del gobierno, diseñado para ofrecer **transporte público masivo y eléctrico por medio de trenes** a los habitantes de la capital y sus zonas conurbadas (Sistema de Transporte Colectivo Metro, n.d.). Este sistema, mejor conocido como *el Metro de la Ciudad de México*, tan solo en el último año movió a más de 1,240 millones de personas a lo largo de sus más de 218 km de extensión. Conformado por **12 líneas**, dos de tipo férreo (Líneas A y 12) y diez de tipo neumático (Líneas 1, 2, 3, 4, 5, 6, 7, 8, 9 y B), que suman un total de **195 estaciones** repartidas a lo largo de la Zona Metropolitana del Valle de México, en enero de 2026 se posicionó como el medio de transporte más utilizado en la ciudad (Once Noticias 2026).

Una encuesta realizada por el STC y Parametría en 2013 revela que las principales razones por las que los usuarios eligen el metro son su **rapidez**, su **bajo costo** y su **nivel de cobertura** (Sistema de Transporte Colectivo Metro 2013). Una encuesta posterior de 2018 confirma esta tendencia: **economía**, **rapidez** y **cercanía** siguen siendo los motivos dominantes de elección (Sistema de Transporte Colectivo Metro 2018). Estos datos señalan que de forma estructural, para una parte importante de la población, el metro no es una opción entre varias, sino la única alternativa viable de transporte. La segunda encuesta, establece que la parte de la población afectada corresponde principalmente a estudiantes y trabajadores independientes, los cuales perciben un ingreso mensual menor a los 7,001 pesos que corresponden a alrededor de 2.5 salarios mínimos mensuales en 2018 Comisión Nacional de los Salarios Mínimos (2017).

Es esta misma demanda masiva la que genera sus propias tensiones, pues como se refleja en las encuestas citadas, entre los problemas más mencionados por los usuarios se encuentran **el elevado nivel de afluencia**, los **retrasos en el servicio** y la presencia de **vendedores ambulantes** (Sistema de Transporte Colectivo Metro 2013). El 63% de las mujeres y el 61% de los hombres encuestados en 2018 calificaron su nivel de satisfacción con el tiempo de traslado como regular o bajo (Sistema de Transporte Colectivo Metro 2018). En otras palabras, el metro es **indispensable y, al mismo tiempo, insatisfactorio** para la mayoría de quienes lo usan. Esta paradoja tiene una causa principal: **la saturación crónica de la red**, un fenómeno que no se distribuye de manera uniforme entre líneas ni entre meses, pero que el sistema hasta ahora ha gestionado sin herramientas que permitan anticiparlo o caracterizarlo sistemáticamente de forma histórica.

Entender los patrones de saturación no es solo un problema operativo, sino una cuestión de equidad. Cuando una línea opera de forma habitual por encima de su capacidad ideal, las consecuencias recaen desproporcionadamente sobre quienes no pueden elegir otro medio.

Identificar qué líneas concentran la mayor demanda, en qué momentos del año se alcanzan niveles críticos y qué perfil de usuario caracteriza cada segmento de la red es, por tanto, información con valor social directo pues permite orientar decisiones de mantenimiento, ajuste de frecuencias en los trenes y asignación de personal en donde más se necesita.

Con ese propósito, el proyecto plantea dos objetivos concretos y complementarios. El primero es **clasificar el nivel de saturación diario de cada estación del STC como bajo, medio, alto o crítico, a partir de sus patrones históricos de afluencia**, mediante algoritmos de clasificación supervisada. El segundo es **segmentar las estaciones de la red según sus perfiles de demanda**, identificando grupos con comportamiento similar a través de técnicas de clustering no supervisado. Ambos análisis buscan responder una pregunta central: ¿es posible anticipar y caracterizar los niveles de demanda del sistema a partir de sus propios registros históricos?

2.1 Descripción del dataset

Para responder esta pregunta el conjunto de datos elegido fue el de **Afluencia Diaria del Metro (Desglosada)**, disponible públicamente en el [Portal de Datos Abiertos](#). Este dataset registra el total de entradas a cada estación de cada línea desagregadas por esquema de pago para el periodo comprendido entre enero de 2021 y abril de 2026, con un total de 1,138,411 registros. Este dataset es pertinente por varias razones: su granularidad diaria por estación permite construir una serie de tiempo lo suficientemente completa para nuestros modelos; su origen oficial garantiza consistencia en la recolección; y su cobertura temporal abarca tanto el periodo de recuperación post-pandemia como años de demanda normalizada, lo que introduce una variabilidad estructural interesante.

```
import pandas as pd

# cargamos el conjunto de datos
df = pd.read_csv("../data/raw/afluenciastc_desglosado_04_2026.csv")

df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 1138410 entries, 0 to 1138409
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0   fecha       1138410 non-null  str
1   mes         1138410 non-null  str
2   anio        1138410 non-null  int64
```

```

3  linea      1138410 non-null  str
4  estacion   1138410 non-null  str
5  tipo_pago  1138410 non-null  str
6  afluencia  1138410 non-null  int64
dtypes: int64(2), str(5)
memory usage: 60.8 MB

```

2.1.1 Diccionario de Datos

También en el [Portal de Datos Abiertos](#) es posible obtener el siguiente diccionario.

No.	Columna	Etiqueta	Descripción	Tipo de Dato	Formato	Valores Permitidos	Campos Vacíos
1	fecha	fecha	Fecha de la medición de afluencia	Fecha	AAAA-MM-DD	Fechas	SIN DATOS VACÍOS
2	ano	año	Año de la medición de afluencia	Número entero	-	Año	SIN DATOS VACÍOS
3	mes	fecha	Mes de la medición de afluencia	Alfanumérico	-	Mes en texto (ej. “Enero”)	SIN DATOS VACÍOS
4	linea	línea	Línea del STC - Metro de la medición de afluencia	Alfanumérico	-	“Línea X”	SIN DATOS VACÍOS

No.	Columna	Etiqueta	Descripción	Tipo de Dato	Formato	Valores Permitidos	Campos Vacíos
5	afluencia_boleto	Entradas al metro por boleto	Total de personas que han ingresado bajo el esquema de pago con boleto	Número entero	-	Número entero	SIN DATOS VACÍOS
6	afluencia_prepago	Entradas al metro por prepago	Total de personas que han ingresado bajo el esquema de prepago	Número entero	-	Número entero	SIN DATOS VACÍOS
7	afluencia_gratuidad	Entradas al metro por gratuidad	Total de personas que han ingresado bajo el esquema de gratuidad	Número entero	-	Número entero	SIN DATOS VACÍOS

No.	Columna	Etiqueta	Descripción	Tipo de Dato	Formato	Valores Permitidos	Campos Vacíos
8	afluencia_entradas	entradas totales al metro	Total de personas que a la fecha han ingresado a la red del Sistema de Transporte Colectivo Metro	Número entero	-	Número entero	SIN DATOS VACÍOS

Nota: Este diccionario describe la estructura de datos para el seguimiento de la afluencia en el Sistema de Transporte Colectivo (STC) Metro de la CDMX.

2.1.2 Consideraciones sobre el tipo de pago

La columna **tipo_pago** merece una explicación particular, ya que sus tres categorías no son estáticas a lo largo del periodo cubierto por los datos.

El **boleto** físico fue la forma de pago tradicional del STC durante décadas, pero su uso se fue descontinuo de forma gradual. La transición comenzó en septiembre de 2023 en las líneas 4 y 6, y el cese total en toda la red se hizo efectivo el 20 de abril de 2024 (Constantino 2024). Cabe mencionar que la línea 12 es un caso especial ya que desde prácticamente su inauguración en 2012 operaba exclusivamente con tarjeta, por lo que sus registros de boleto son históricamente bajos o nulos.

El **prepago** corresponde al acceso mediante la Tarjeta de Movilidad Integrada, y es actualmente la modalidad principal de la red.

La **gratuidad** cubre el acceso sin costo para adultos mayores, personas con discapacidad y niños menores de 5 años, conforme a los requisitos establecidos por el organismo. Al tratarse de una población específica, esperadamente su volumen es estructuralmente menor al de las otras dos modalidades, aunque puede variar entre líneas dependiendo de las características demográficas de las zonas que atraviesan.

2.1.3 Limitaciones del conjunto de datos

El dataset presenta dos limitaciones relevantes para el análisis. La primera es la **ausencia de datos sobre capacidad instalada** por línea o por estación, lo que impide calcular niveles de ocupación reales respecto a un máximo definido; los niveles de saturación que se construirán en este proyecto se basarán por tanto en referencias históricas relativas, no en umbrales absolutos de capacidad. La segunda es la **falta de contexto operativo**: eventos como trabajos de remodelación, cierres temporales de estaciones o la presencia de eventos masivos en zonas aledañas afectan directamente la afluencia registrada, pero no están documentados en el dataset.

3 Limpieza del dataset

```
import sys
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
sys.path.append('../..')

from src.limpieza import cargar_datos, limpiar_dataframe, normalizar_str, filtrar_cierres
from src.preparacion import guardar_datos, pivot_tipo_pago, agregar_variable_objetivo, agregar_variable_objetivo

df = cargar_datos('../..data/raw/afluenciastc_desglosado_04_2026.csv')

df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 1138410 entries, 0 to 1138409
Data columns (total 7 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   fecha       1138410 non-null  str
 1   mes         1138410 non-null  str
 2   anio        1138410 non-null  int64
 3   linea       1138410 non-null  str
 4   estacion    1138410 non-null  str
 5   tipo_pago   1138410 non-null  str
 6   afluencia   1138410 non-null  int64
dtypes: int64(2), str(5)
memory usage: 60.8 MB
```

3.1 Valores registrados

Para comenzar con el tratamiento de los datos, vamos a revisar los valores registrados en cada columna del dataset, con el fin de identificar puntos de mejora en la calidad de los datos que podemos abordar.

```

# por cada columna categórica (a excepción de la fecha de medición y la estación) obtenemos
columnas = ['mes', 'anio', 'linea', 'tipo_pago']

fig = make_subplots(rows=2, cols=2, subplot_titles=columnas)

for i, col in enumerate(columnas):
    row = i // 2 + 1
    col_pos = i % 2 + 1

    frecuencias = df[col].value_counts().reset_index()
    frecuencias.columns = [col, 'frecuencia']

    fig.add_trace(
        go.Bar(x=frecuencias[col].astype(str), y=frecuencias['frecuencia'], name=col),
        row=row, col=col_pos
    )

fig.update_layout(height=700, showlegend=False, title_text='Frecuencias de variables categóricas')
fig.show()

```

Unable to display output for mime type(s): text/html

Unable to display output for mime type(s): text/html

De este primer acercamiento, podemos notar que: * Los meses se registran correctamente pues los meses con mayor número de días o que se registran **completamente** en los 6 años del dataset son los más frecuentes, mientras que los meses de mayo a diciembre, que no se presentan en los registros del año 2026 son menos frecuentes * Existe un duplicado en las líneas debido a la presencia de acentos mal codificados * En efecto se tienen únicamente tres tipos de pago válidos

3.2 Limpieza de acentos

Como vimos las columnas `linea` y `estacion` tienen problemas debido a la presencia de acentos mal codificados, por ello, lo que se hizo primero fue eliminar el prefijo “Linea” de la respectiva columna y reparar los errores de codificación en las estaciones.

```
df = limpiar_dataframe(df)

print(f'Líneas únicas: {len(df['linea'].unique())}')
print(f'Estaciones únicas: {len(df['estacion'].unique())}')
```

Líneas únicas: 12
Estaciones únicas: 165

Una vez hecho esto para simplificar nuestros datos se decidió eliminar los acentos y pasar a mayúsculas nuestros datos.

```
df = normalizar_str(df)

print(f'Estaciones únicas: {len(df['estacion'].unique())}')
```

Estaciones únicas: 163

Gracias a esta normalización, se logró tener un registro correcto de las 12 líneas y de las 163 estaciones sin repetir aquellas que forman parte de más de una línea.

```
fig2 = make_subplots(rows=2, cols=2, subplot_titles=columnas)

for i, col in enumerate(columnas):
    row = i // 2 + 1
    col_pos = i % 2 + 1

    frecuencias = df[col].value_counts().reset_index()
    frecuencias.columns = [col, 'frecuencia']

    fig2.add_trace(
        go.Bar(x=frecuencias[col].astype(str), y=frecuencias['frecuencia'], name=col),
        row=row, col=col_pos
    )

fig2.update_layout(height=700, showlegend=False, title_text='Frecuencias de variables categóricas')
fig2.show()
```

Unable to display output for mime type(s): text/html

3.3 Integridad de fechas

El siguiente paso es revisar que nuestras fechas se procesan de manera adecuada y que en efecto el periodo reportado en el portal es correcto.

```
print(f'Inicio: {df['fecha'].min()}')  
print(f'Fin: {df['fecha'].max()}')
```

Inicio: 2021-01-01 00:00:00

Fin: 2026-04-30 00:00:00

3.4 Datos duplicados

Dado que una estación puede pertenecer a distintas líneas, consideramos un registro duplicado como aquel que tiene la misma fecha de medición, línea, estación y método de pago. Gracias a la siguiente línea es fácil ver que en este caso no se tienen datos duplicados.

```
df.duplicated(subset=['fecha', 'linea', 'estacion', 'tipo_pago']).sum()
```

```
np.int64(0)
```

4 Preparación del dataset

Para que nuestros modelos sean capaces de identificar el periodo a partir del cual ya no se permite el acceso por boleto, añadimos una columna `post_boleto` para indicar si la medición se hizo antes (0) o después (1) del fin del uso de boletos.

Originalmente nuestro conjunto tiene 3 filas muy parecidas por cada fecha pues cada una registra un método de pago distinto. Sin embargo, para reducir el número de filas y simplificar los pasos posteriores, lo que haremos es unir los distintos métodos de pago en un mismo registro **sin perder granularidad** como se muestra a continuación.

```
df_simplificado = pivot_tipo_pago(df)

print(f'Filas y columnas: {df_simplificado.shape}')
```

Filas y columnas: (379470, 10)

Ahora que tenemos una columna específica para la afluencia total, podemos ver un resumen de las afluencias registradas en el dataset.

```
print(df_simplificado['afluencia_total'].describe())
```

```
count    379470.000000
mean      15216.383287
std       14497.793993
min         0.000000
25%        6079.000000
50%       11853.000000
75%       19701.000000
max      139988.000000
Name: afluencia_total, dtype: float64
```

Gracias a esto notamos que existen estaciones con afluencia 0 por lo que es importante revisar a que líneas corresponden estos registros.


```
ceros = df_simplificado[df_simplificado['afluencia_total'] == 0]
print(f"Registros con afluencia total 0: {len(ceros)}")
print(ceros['linea'].value_counts())
```

Registros con afluencia total 0: 29530

```
linea
12    15570
1     11381
2         985
9         826
3         526
7         124
5          39
6          34
4          32
B          12
8           1
Name: count, dtype: int64
```

Como vemos, los registros de afluencia cero están asociados mayormente a las líneas 1 y 12. Recordemos que la línea 12 estuvo cerrada debido al colapso en el tramo elevado ocurrido el 3 de mayo de 2021 y no volvió a operar en su totalidad hasta el 30 de enero de 2024 (Sarabia 2024), mientras que la línea 1 estuvo en remodelación del 9 de julio de 2022 al 16 de noviembre de 2025 (El Financiero 2025). Esto explica en buena medida por qué estas líneas registran tantos días con afluencia de cero.

No obstante, contemplar estos datos introduce un sesgo en el análisis, pues esa afluencia de cero no refleja una ausencia de demanda real sino una demanda que se vio obligada a redistribuirse en otros medios de transporte, como el sistema auxiliar de RTP que también resultó ineficiente. Es por esta razón que se excluyeron únicamente los registros con afluencia total igual a cero dentro de los periodos de cierre documentados de ambas líneas, respetando así los registros válidos de estaciones que sí permanecieron en operación durante esos mismos periodos.

```
df_modelo = filtrar_cierres(df_simplificado)
print(df_modelo.shape)

# Guardar dataset limpio para EDA y modelado
df_modelo.to_csv("../data/processed/df_modelo.csv", index=False)
```

(358086, 10)

4.1 Modelo para clasificación

4.1.1 Variable objetivo

Para desarrollar nuestro modelo de clasificación necesitamos una variable objetivo pues será la base de nuestro algoritmo de aprendizaje supervisado. En este caso, dado que nos interesa **clasificar el nivel de saturación diario** pero desconocemos el valor exacto de la capacidad instalada en cada estación, aprovechamos los registros históricos para calcular los percentiles de afluencia de cada estación a lo largo del tiempo. Este cálculo se realiza de forma individual por estación y no sobre el dataset completo, con el fin de que cada estación se compare contra su propio historial y no contra estaciones de mayor o menor tamaño.

Así, por cada registro, si la `afluencia_total` es menor o igual al percentil 25 de su estación, se clasifica como nivel bajo (0); si está por encima del percentil 25 pero por debajo o igual al percentil 60, como nivel medio (1); si supera el percentil 60 pero no el percentil 85, como nivel alto (2); y si está por encima del percentil 85, como nivel crítico (3). De esta forma, los casos críticos corresponden aproximadamente al 15% superior del historial de cada estación.

```
df_modelo = agregar_variable_objetivo(df_modelo)
```

```
frecuencias = df_modelo['nivel_saturacion'].value_counts().sort_index().reset_index()
frecuencias.columns = ['nivel', 'frecuencia']
frecuencias['nivel'] = frecuencias['nivel'].map({
    0: 'Bajo',
    1: 'Medio',
    2: 'Alto',
    3: 'Crítico'
})

fig = px.bar(
    frecuencias,
    x='nivel',
    y='frecuencia',
    title='Distribución de niveles de saturación',
    color='nivel',
    color_discrete_map={
        'Bajo': '#2ecc71',
        'Medio': '#f1c40f',
        'Alto': '#e67e22',
        'Crítico': '#e74c3c'
    }
)
```


Estructura de las nuevas características integradas:

	dia_semana	es_festivo	es_quincena	lluvia_historica
0	4	0	0	0
1	4	0	0	0
2	4	0	0	0

Excluimos el conteo de ‘afluencia_total’ para que el modelo no dependa de esta variable, pues aunque es un buen indicador del nivel de saturación, no es una variable que se pueda conocer a priori en un día específico, lo que limita la utilidad práctica del modelo. En cambio, las variables que hemos incluido permiten al modelo aprender patrones relacionados con el contexto temporal y sociocultural que pueden influir en la afluencia sin depender directamente de la medición de afluencia total del día.

Así, el dataset que tenemos para el modelo de clasificación es el siguiente:

```
df_modelo.head(3)

columnas_clas = [
    'linea_encoded',
    'estacion_encoded',
    'dia_semana',
    'mes_encoded',
    'anio',
    'es_festivo',
    'es_quincena',
    'lluvia_historica',
    'post_boleto',
    'nivel_saturacion'
]

df_clasificacion = df_modelo[columnas_clas].copy()
guardar_datos(df_clasificacion, "../../data/processed/df_clasificacion.csv")

print(f"Dimensiones finales del dataset exportado: {df_clasificacion.shape[0]:,} registros × {df_clasificacion.shape[1]} columnas")
df_clasificacion.head(3)
```

Dimensiones finales del dataset exportado: 358,086 registros × 10 columnas

	linea_encoded	estacion_encoded	dia_semana	mes_encoded	anio	es_festivo	es_quincena	lluvia_historica
0	0	10	4	3	2021	0	0	0
1	0	11	4	3	2021	0	0	0

	linea_encoded	estacion_encoded	dia_semana	mes_encoded	anio	es_festivo	es_quincena	lluvia
2	0	16	4	3	2021	0	0	0

4.2 Modelo para clustering

En el caso del dataset para el modelo de clustering, lo que nos interesa es que cada fila describa el comportamiento histórico de cada estación, pues de esta forma el modelo puede segmentar las estaciones de la red según sus perfiles de demanda, en lugar de las mediciones diarias.

Por ello, en este caso lo que hicimos fue calcular la proporción que representa cada tipo de pago en la afluencia total de cada estación. Luego agregamos los datos por estación, calculando la media y la desviación estándar de la afluencia y el nivel de saturación por cada estación.

```
df_clustering = construir_df_clustering(df_modelo)
guardar_datos(df_clustering, "../../../data/processed/df_clustering.csv")

print(df_clustering.shape)
df_clustering.head(3)
```

(195, 7)

	linea	estacion	afluencia_media	afluencia_std	proporcion_prepago	proporcion_sat
0	1	BALBUENA	6826.178790	1972.007982	0.665244	0.160
1	1	BALDERAS	14966.389778	10277.999962	0.595050	0.238
2	1	BOULEVARD PUERTO AEREO	14628.169273	4195.615651	0.710051	0.187

5 EDA — Análisis Exploratorio de Datos

El objetivo de esta sección es analizar el comportamiento histórico de la afluencia en las estaciones del Sistema de Transporte Colectivo Metro antes de aplicar modelos de clasificación y clustering.

Trabajaremos sobre el dataset limpio `df_modelo.csv`, que contiene registros diarios por estación en el periodo comprendido entre **1 de enero de 2021 y 30 de abril de 2026**.

5.1 Carga del Dataset

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from matplotlib.ticker import FuncFormatter

# Cargar dataset limpio
df = pd.read_csv("../data/processed/df_modelo.csv")

# Convertir fecha a datetime
df["fecha"] = pd.to_datetime(df["fecha"])

# Mostrar únicamente estructura del dataset
df.info()

# Verificar rango temporal
print("Rango temporal del dataset:")
print("Inicio:", df["fecha"].min())
print("Fin:", df["fecha"].max())
```

```
<class 'pandas.DataFrame'>
RangeIndex: 358086 entries, 0 to 358085
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
```

```

---  -----
0  fecha                358086 non-null  datetime64[us]
1  linea                358086 non-null  str
2  estacion            358086 non-null  str
3  mes                 358086 non-null  str
4  anio                358086 non-null  int64
5  post_boleto         358086 non-null  int64
6  afluencia_boleto    358086 non-null  int64
7  afluencia_gratuidad 358086 non-null  int64
8  afluencia_prepago   358086 non-null  int64
9  afluencia_total     358086 non-null  int64
dtypes: datetime64[us](1), int64(6), str(3)
memory usage: 27.3 MB
Rango temporal del dataset:
Inicio: 2021-01-01 00:00:00
Fin: 2026-04-30 00:00:00

```

5.2 Distribución de la Afluencia Total (2021–2026)

La variable central de nuestro análisis es `afluencia_total`, que representa el número total de personas que ingresaron a una estación en un día determinado.

A continuación observamos su distribución en todo el periodo 2021–2026:

```

plt.figure(figsize=(10,6))

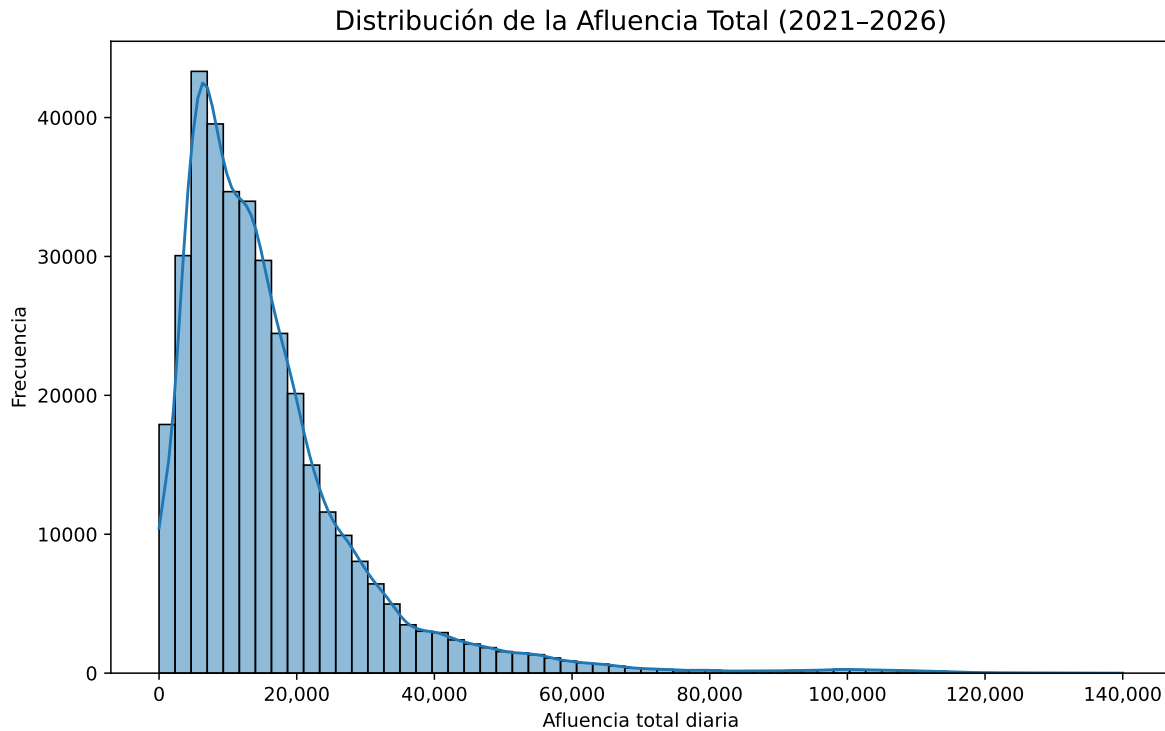
sns.histplot(df["afluencia_total"], bins=60, kde=True)

plt.title("Distribución de la Afluencia Total (2021-2026)", fontsize=14)
plt.xlabel("Afluencia total diaria")
plt.ylabel("Frecuencia")

# Formato con separador de miles
plt.gca().xaxis.set_major_formatter(FuncFormatter(lambda x, _: f"{int(x):,}"))

plt.show()

```



5.3 Interpretación Inicial

A partir del histograma y las estadísticas observadas previamente:

- La mayoría de los registros diarios se concentran entre aproximadamente **5,000 y 25,000 pasajeros**.
- Existen valores extremos que superan los **100,000 pasajeros diarios**, lo que indica estaciones de altísima demanda.
- La mediana es considerablemente menor que el valor máximo, lo que confirma la presencia de una cola larga.

Esto sugiere que la red presenta una fuerte heterogeneidad: la mayoría de las estaciones operan en rangos moderados de afluencia, mientras que unas pocas concentran niveles extraordinariamente altos de demanda.

En la siguiente sección analizaremos cómo se distribuye esta afluencia por línea para identificar patrones estructurales dentro de la red.

5.4 Detección de Valores Atípicos

Para identificar posibles valores extremos en la variable `afluencia_total`, utilizamos un boxplot basado en la regla del rango intercuartílico (IQR).

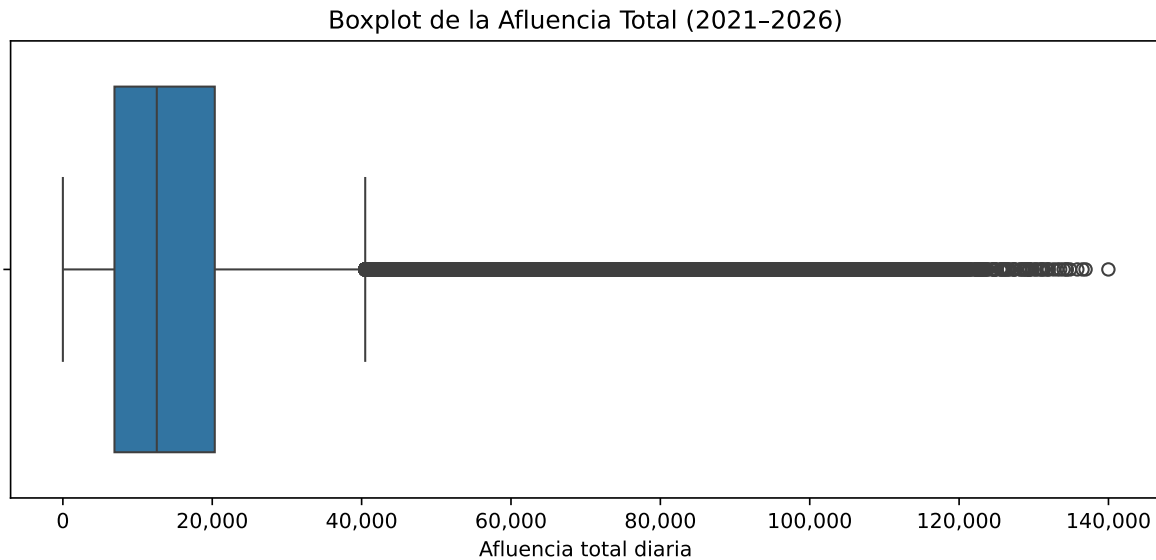
```
plt.figure(figsize=(10,4))

sns.boxplot(x=df["afluencia_total"])

plt.title("Boxplot de la Afluencia Total (2021-2026)")
plt.xlabel("Afluencia total diaria")

plt.gca().xaxis.set_major_formatter(FuncFormatter(lambda x, _: f"{int(x):,}"))

plt.show()
```



5.4.1 Interpretación

El boxplot confirma la presencia de valores atípicos superiores.

Estos registros no representan errores, sino estaciones con alta concentración de demanda.

Por tanto, no se eliminan, ya que reflejan comportamiento real del sistema y son relevantes para el modelado de saturación.

5.5 Afluencia Promedio por Línea

Ahora analizaremos cómo se distribuye la demanda entre las distintas líneas del sistema.

Calcularemos:

- Media de afluencia total por línea
- Desviación estándar por línea
- Las 5 líneas con mayor promedio de afluencia

```
# Agrupar por línea
linea_stats = df.groupby("línea")["afluencia_total"].agg(["mean", "std"]).reset_index()

# Renombrar columnas
linea_stats.columns = ["línea", "afluencia_media", "afluencia_std"]

linea_stats.sort_values("afluencia_media", ascending=False)
```

	línea	afluencia_media	afluencia_std
2	2	21029.403049	16513.730295
3	3	20723.211398	18032.286907
10	A	19424.464029	10848.876986
9	9	19252.991221	16474.654630
8	8	16428.942852	18435.285543
0	1	16309.100224	11903.115833
11	B	15436.479788	11221.492161
7	7	14715.302452	10326.582367
1	12	14177.148480	11085.801711
5	5	11644.394853	12635.442144
6	6	8834.725544	7119.350587
4	4	6356.470041	4878.477017

5.5.1 Visualización de la Afluencia Media por Línea

```
plt.figure(figsize=(10,6))

sns.barplot(

    data=linea_stats.sort_values("afluencia_media", ascending=False),
```

```

x="línea",

y="afluencia_media"

)

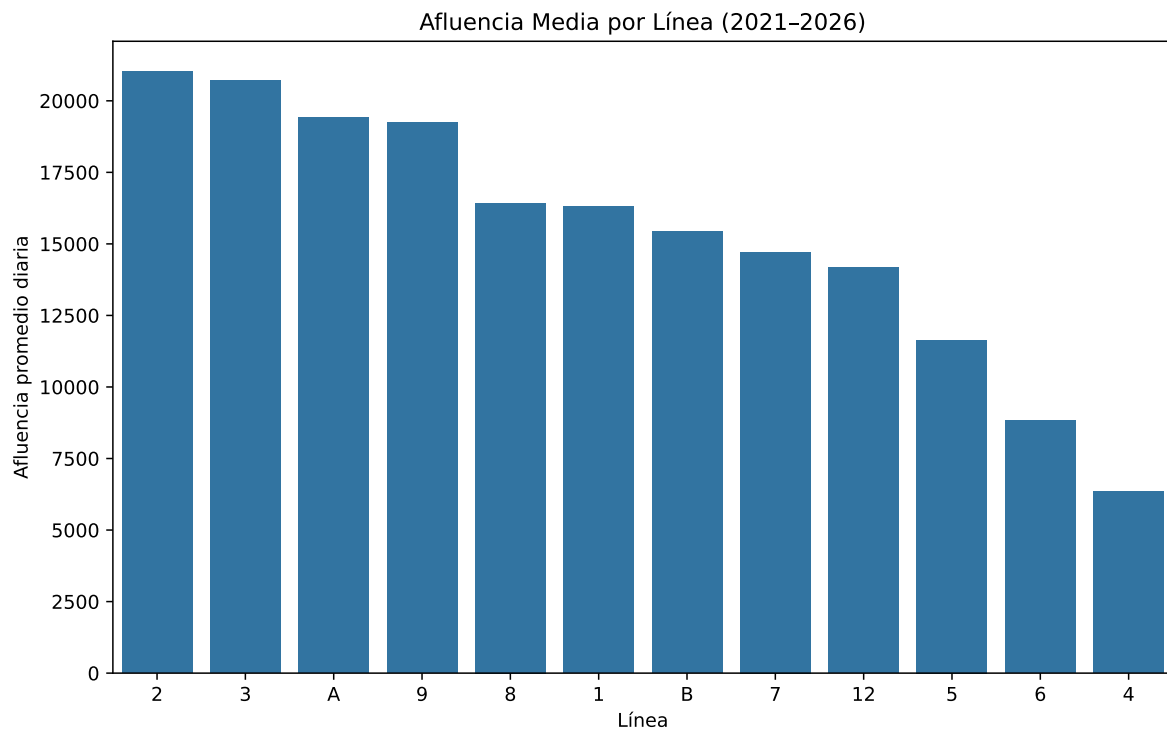
plt.title("Afluencia Media por Línea (2021-2026)")

plt.xlabel("Línea")

plt.ylabel("Afluencia promedio diaria")

plt.show()

```



5.5.2 Top 5 Líneas con Mayor Afluencia Promedio

```

top5 = linea_stats.sort_values("afluencia_media", ascending=False).head(5)
top5

```

	linea	afluencia_media	afluencia_std
2	2	21029.403049	16513.730295
3	3	20723.211398	18032.286907
10	A	19424.464029	10848.876986
9	9	19252.991221	16474.654630
8	8	16428.942852	18435.285543

5.6 Interpretación

La Línea 2 presenta la mayor afluencia promedio diaria (21,029 pasajeros), seguida muy de cerca por la Línea 3 (20,723). Esto indica que ambas concentran la mayor demanda estructural del sistema.

La Línea A y la Línea 9 también muestran niveles elevados de afluencia promedio, lo que sugiere que funcionan como corredores de alta movilidad dentro de la red.

En contraste, las Líneas 4 y 6 presentan los promedios más bajos, con valores cercanos a 6,356 y 8,834 pasajeros diarios respectivamente, lo que indica una demanda estructural considerablemente menor.

En términos de variabilidad, la Línea 3 presenta una desviación estándar elevada (18,032), lo que sugiere fluctuaciones importantes en su demanda diaria. Esto puede estar asociado a variaciones estacionales, eventos específicos o cambios operativos.

Este análisis confirma que la demanda no se distribuye de manera uniforme en la red, sino que existen líneas estructuralmente más saturadas que otras.

5.7 Evolución Temporal de la Afluencia (2021–2026)

Para analizar cómo ha cambiado la demanda del sistema a lo largo del tiempo, calculamos el promedio diario anual de afluencia. Es decir, para cada año tomamos todos los días registrados de todas las estaciones y obtenemos el promedio de pasajeros diarios.

```
# Calcular afluencia promedio anual
afluencia_anual = (
    df.groupby("anio")["afluencia_total"]
    .mean()
    .reset_index()
)

afluencia_anual
```

	anio	afluencia_total
0	2021	11974.064204
1	2022	16677.324380
2	2023	17379.572266
3	2024	16450.377801
4	2025	17637.802487
5	2026	17402.740427

5.7.1 Visualización de la Evolución Anual

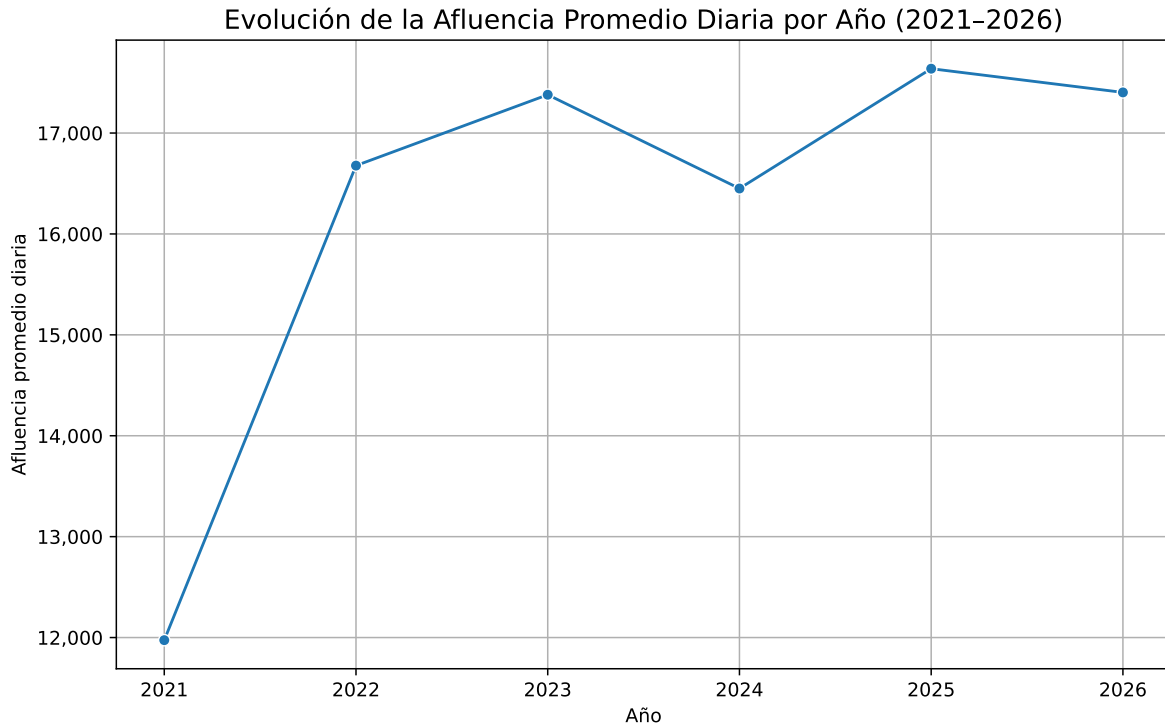
```
plt.figure(figsize=(10,6))

sns.lineplot(
    data=afluencia_anual,
    x="anio",
    y="afluencia_total",
    marker="o"
)

plt.title("Evolución de la Afluencia Promedio Diaria por Año (2021-2026)", fontsize=14)
plt.xlabel("Año")
plt.ylabel("Afluencia promedio diaria")

plt.gca().yaxis.set_major_formatter(FuncFormatter(lambda x, _: f"{int(x):,}"))

plt.grid(True)
plt.show()
```



5.8 Interpretación Temporal

El análisis temporal permite observar la dinámica estructural del sistema:

- En 2021 se observan niveles aún afectados por el periodo post-pandemia.
- A partir de 2022 y 2023 se aprecia una recuperación progresiva de la demanda.
- Los años más recientes (2024–2025) muestran una estabilización en niveles elevados de afluencia.
- El año 2026 presenta datos parciales (hasta abril), por lo que debe interpretarse con cautela.

Estos resultados muestran que la demanda del sistema no se mantiene constante a lo largo del tiempo. Cambia dependiendo del contexto general de la ciudad, como la recuperación posterior a la pandemia, modificaciones operativas o cambios en el sistema de pago.

5.9 Comparación de Afluencia: Antes y Después del Fin del Boleto Físico

La variable `post_boleto` indica si el registro corresponde a un periodo:

- 0 → Antes del 20 de abril de 2024 (uso de boleto físico)
- 1 → Después del 20 de abril de 2024 (solo tarjeta)

Para analizar si hubo un cambio significativo en la demanda, calculamos el promedio diario en ambos periodos.

```
afluencia_boleto_periodo = (
    df.groupby("post_boleto")["afluencia_total"]
    .mean()
    .reset_index()
)

afluencia_boleto_periodo["post_boleto"] = afluencia_boleto_periodo["post_boleto"].map({
    0: "Antes del fin del boleto",
    1: "Después del fin del boleto"
})

afluencia_boleto_periodo
```

	post_boleto	afluencia_total
0	Antes del fin del boleto	15322.651362
1	Después del fin del boleto	17311.190969

5.9.1 Visualización Comparativa

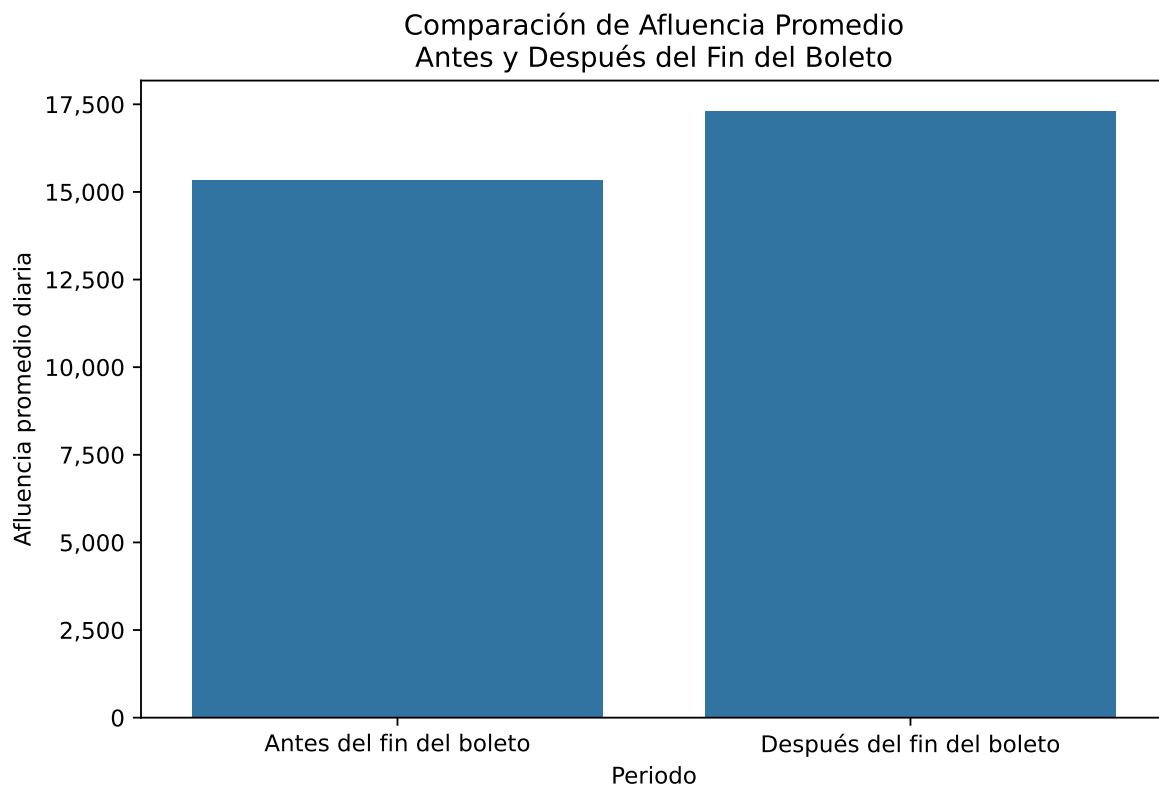
```
plt.figure(figsize=(8,5))

sns.barplot(
    data=afluencia_boleto_periodo,
    x="post_boleto",
    y="afluencia_total"
)

plt.title("Comparación de Afluencia Promedio\nAntes y Después del Fin del Boleto")
plt.xlabel("Periodo")
plt.ylabel("Afluencia promedio diaria")

plt.gca().yaxis.set_major_formatter(FuncFormatter(lambda x, _: f"{int(x):,}"))

plt.show()
```



5.10 Interpretación

Se observa que la afluencia promedio diaria es mayor en el periodo posterior al fin del boleto físico.

Sin embargo, este incremento no puede atribuirse únicamente al cambio en el método de pago. El periodo “antes del fin del boleto” incluye los años 2021 y parte de 2022, que todavía reflejan efectos posteriores a la pandemia, cuando la movilidad urbana se encontraba reducida.

Por el contrario, el periodo posterior coincide con una etapa de recuperación económica y normalización de actividades presenciales, lo que naturalmente incrementa la demanda del sistema.

Por tanto, el aumento observado parece responder más a una recuperación estructural de la movilidad que al efecto directo de eliminar el boleto físico.

5.11 Evolución de la Movilidad por Mes (2021–2026)

Para identificar patrones estacionales en la movilidad del sistema, analizamos la afluencia promedio diaria por mes considerando todo el periodo 2021–2026.

Calculamos el promedio diario de afluencia para cada mes agregando todos los años y todas las estaciones.

```
# Promedio mensual global
afluencia_mensual = (
    df.groupby("mes")["afluencia_total"]
      .mean()
      .reset_index()
)

# Ordenar meses cronológicamente
orden_meses = [
    "ENERO", "FEBRERO", "MARZO", "ABRIL", "MAYO", "JUNIO",
    "JULIO", "AGOSTO", "SEPTIEMBRE", "OCTUBRE", "NOVIEMBRE", "DICIEMBRE"
]

afluencia_mensual["mes"] = pd.Categorical(
    afluencia_mensual["mes"],
    categories=orden_meses,
    ordered=True
)

afluencia_mensual = afluencia_mensual.sort_values("mes")

afluencia_mensual
```

	mes	afluencia_total
3	ENERO	13974.408311
4	FEBRERO	15572.892980
7	MARZO	15919.072130
0	ABRIL	15586.025495
8	MAYO	16085.939538
6	JUNIO	16057.184868
5	JULIO	15661.529843
1	AGOSTO	16480.696739
11	SEPTIEMBRE	16924.000074

	mes	afluencia_total
10	OCTUBRE	17664.707602
9	NOVIEMBRE	17797.322004
2	DICIEMBRE	16573.544095

5.11.1 Visualización Mensual

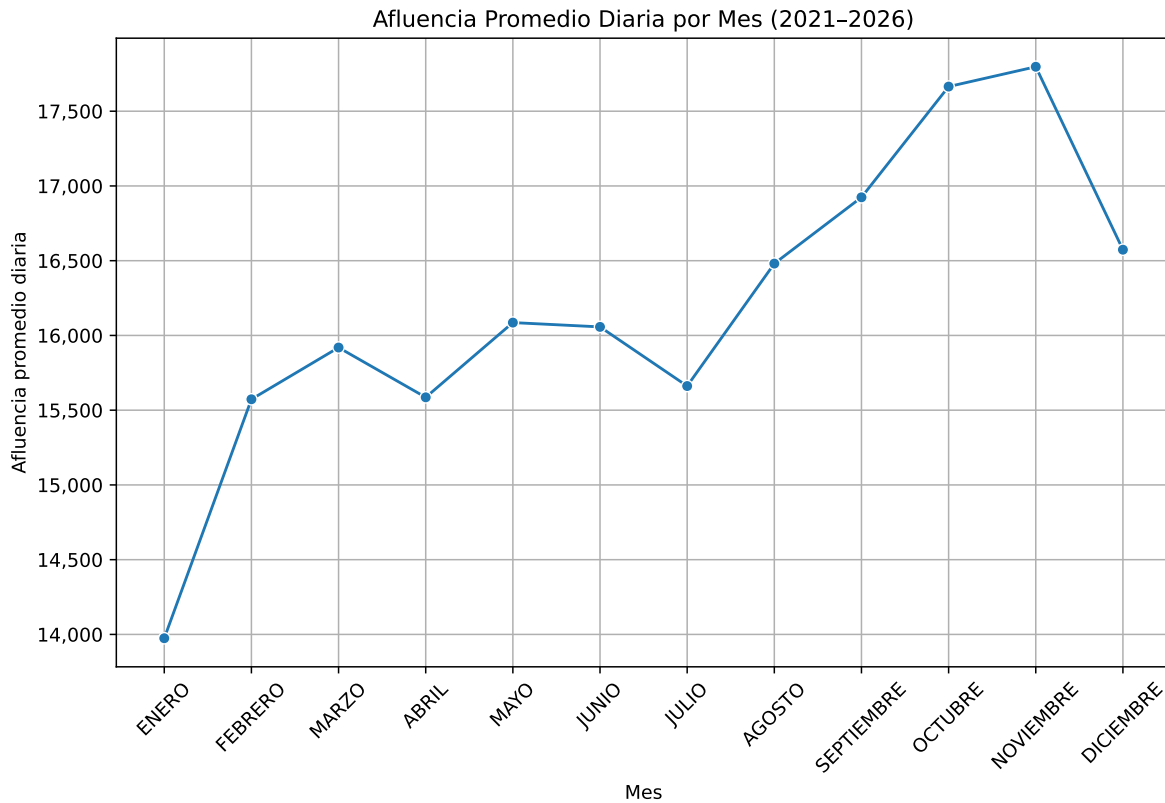
```
plt.figure(figsize=(10,6))

sns.lineplot(
    data=afluencia_mensual,
    x="mes",
    y="afluencia_total",
    marker="o"
)

plt.title("Afluencia Promedio Diaria por Mes (2021-2026)")
plt.xlabel("Mes")
plt.ylabel("Afluencia promedio diaria")

plt.gca().yaxis.set_major_formatter(FuncFormatter(lambda x, _: f"{int(x):,}"))

plt.xticks(rotation=45)
plt.grid(True)
plt.show()
```



5.12 Interpretación Mensual

El análisis mensual permite identificar patrones de estacionalidad en la movilidad urbana.

Se observan disminuciones en meses como julio o diciembre, esto puede estar asociado a periodos vacacionales escolares.

Meses con mayor afluencia podrían coincidir con ciclos laborales y académicos regulares.

Este comportamiento confirma que la movilidad en la ciudad presenta variaciones sistemáticas a lo largo del año, lo cual es relevante para anticipar periodos de mayor saturación.

5.13 Evolución Temporal: Línea 2 vs Línea 3

Para comparar el comportamiento de las líneas con mayor demanda estructural, analizamos la evolución de la afluencia promedio anual de la Línea 2 y la Línea 3.

```

# Filtrar únicamente Línea 2 y Línea 3
lineas_interes = ["2", "3"]

df_filtrado = df[df["linea"].isin(lineas_interes)]

# Calcular promedio anual por línea
evolucion_lineas = (
    df_filtrado.groupby(["anio", "linea"])["afluencia_total"]
    .mean()
    .reset_index()
)

evolucion_lineas

```

	anio	linea	afluencia_total
0	2021	2	12960.211530
1	2021	3	14029.299935
2	2022	2	20222.411073
3	2022	3	21412.470450
4	2023	2	22696.337215
5	2023	3	22492.396477
6	2024	2	24161.320014
7	2024	3	22419.437289
8	2025	2	24772.190411
9	2025	3	22839.158252
10	2026	2	22020.878125
11	2026	3	21996.597222

5.13.1 Visualización Comparativa

```

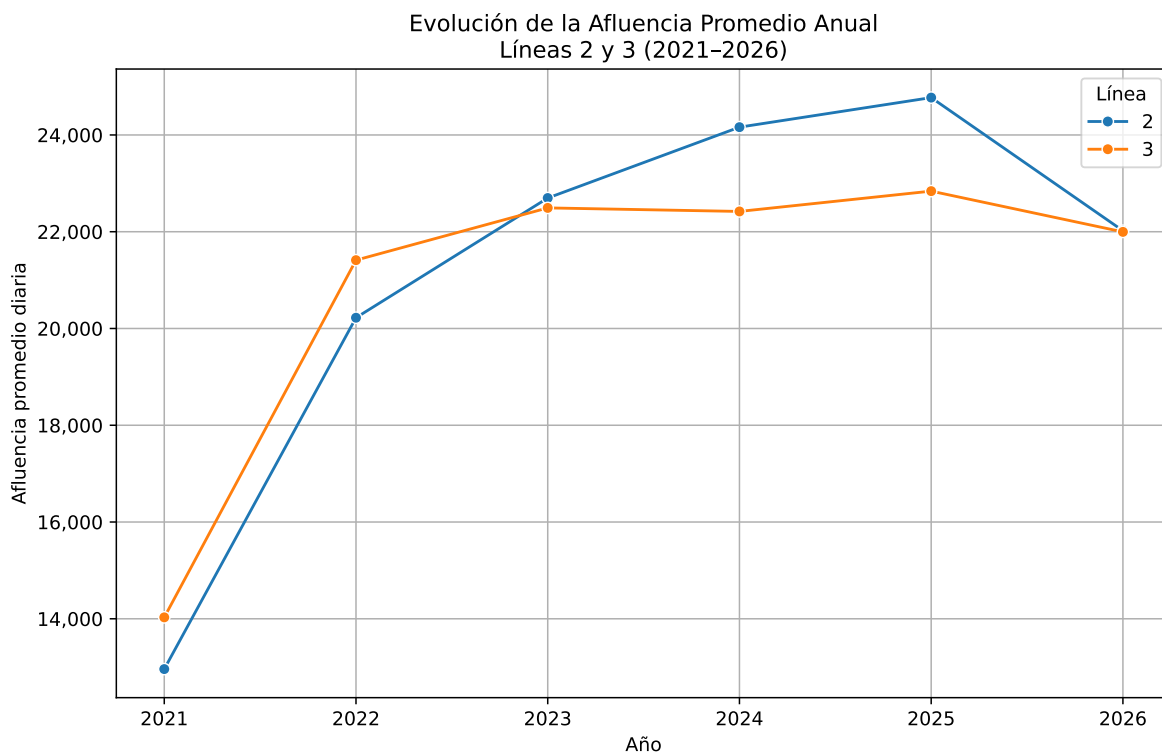
plt.figure(figsize=(10,6))

sns.lineplot(
    data=evolucion_lineas,
    x="anio",
    y="afluencia_total",
    hue="linea",
    marker="o"
)

```

```
plt.title("Evolución de la Afluencia Promedio Anual\nLíneas 2 y 3 (2021-2026)")
plt.xlabel("Año")
plt.ylabel("Afluencia promedio diaria")

plt.gca().yaxis.set_major_formatter(FuncFormatter(lambda x, _: f"{int(x):,}"))
plt.legend(title="Línea")
plt.grid(True)
plt.show()
```



5.14 Interpretación Comparativa

La Línea 2 y la Línea 3 presentan niveles de demanda elevados a lo largo de todo el periodo analizado.

En 2021 se observan valores relativamente menores, lo cual es consistente con el contexto posterior a la pandemia. A partir de 2022 y 2023 se aprecia una recuperación progresiva en ambas líneas.

La Línea 2 mantiene consistentemente una afluencia promedio ligeramente superior a la Línea 3, lo que confirma su posición como la línea con mayor demanda estructural del sistema.

Asimismo, ambas líneas muestran trayectorias similares en el tiempo, lo que sugiere que responden de forma comparable a los cambios generales en la movilidad urbana de la ciudad.

5.15 Relaciones entre Componentes de Afluencia

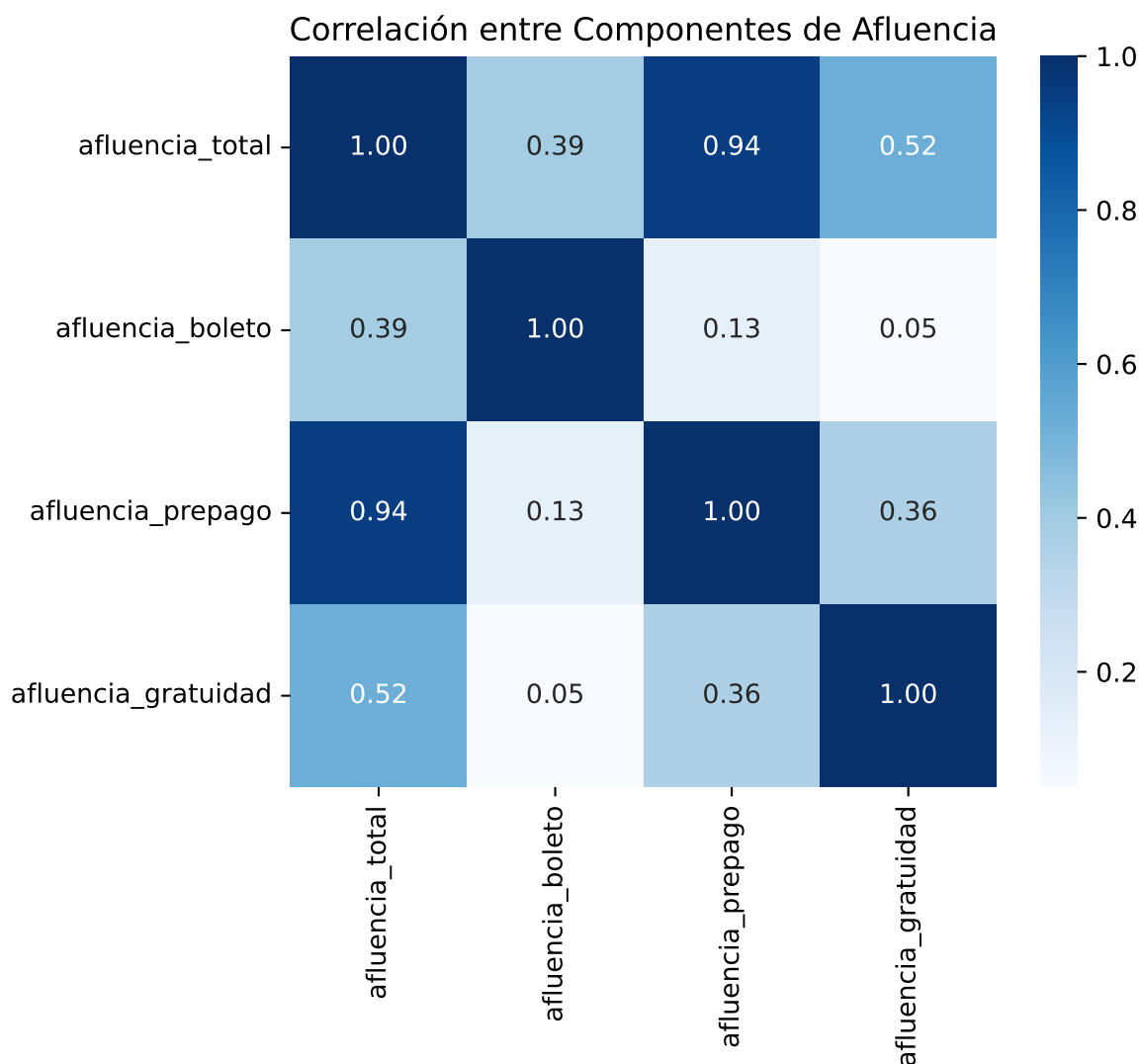
Para entender cómo se compone la demanda diaria, analizamos la correlación entre los distintos métodos de pago y la afluencia total.

```
plt.figure(figsize=(6,5))

corr_pago = df[[
    "afluencia_total",
    "afluencia_boleto",
    "afluencia_prepago",
    "afluencia_gratuidad"
]].corr()

sns.heatmap(corr_pago, annot=True, cmap="Blues", fmt=".2f")

plt.title("Correlación entre Componentes de Afluencia")
plt.show()
```



5.15.1 Interpretación

Se observa una correlación positiva muy alta entre **afluencia_total** y **afluencia_prepago**, lo que indica que el prepago representa la mayor proporción estructural de ingresos al sistema.

La correlación con **afluencia_boleto** es menor en los años recientes, consistente con su eliminación progresiva.

La variable **afluencia_gratuidad** muestra una relación positiva moderada, reflejando que su volumen crece junto con la demanda general, pero en menor proporción.

Este análisis confirma que la dinámica del sistema está fuertemente impulsada por el método de prepago, que constituye el componente dominante de la movilidad diaria.

6 Conclusión General del Análisis Exploratorio

El análisis exploratorio del periodo comprendido entre el **1 de enero de 2021 y el 30 de abril de 2026** permite caracterizar cuantitativamente la magnitud y estructura de la demanda del Sistema de Transporte Colectivo Metro.

Durante este periodo se registraron aproximadamente:

```
print(f"Afluencia total acumulada en el periodo: {df['afluencia_total'].sum():,}")
```

Afluencia total acumulada en el periodo: 5,774,160,966

Es decir, miles de millones de accesos acumulados en poco más de cinco años, lo que confirma la escala masiva del sistema.

En términos promedio:

- La **afluencia media diaria por estación** es de aproximadamente:

```
print(f"Afluencia promedio diaria por estación: {df['afluencia_total'].mean():,.0f}")
```

Afluencia promedio diaria por estación: 16,125

- La mediana es menor que la media, lo que confirma la existencia de una distribución asimétrica con cola hacia valores altos.
- Existen registros que superan los 100,000 pasajeros diarios, lo que evidencia estaciones con altísima concentración de demanda.

A nivel estructural:

- La **Línea 2** y la **Línea 3** concentran la mayor demanda promedio del sistema.
- Las líneas 4 y 6 presentan niveles considerablemente menores.
- La desviación estándar elevada en líneas como la 3 indica fluctuaciones importantes en su demanda diaria.

En el análisis temporal:

- Se observa una recuperación y estabilización progresiva de la movilidad.

- La demanda no es constante en el tiempo, sino dinámica.
- Existen patrones estacionales claros, con disminuciones en meses vacacionales y mayor concentración en meses laborales.

La comparación antes y después del fin del boleto físico muestra un aumento en la afluencia promedio diaria, aunque este fenómeno debe interpretarse como parte de una tendencia general de normalización y crecimiento de la movilidad.

En conjunto, el EDA confirma tres hallazgos clave:

1. La demanda del sistema es altamente heterogénea entre líneas.
2. Existe variabilidad temporal significativa.
3. La distribución de afluencia presenta asimetría y valores extremos relevantes.

Estos resultados justifican plenamente el uso de modelos de clasificación para identificar niveles de saturación y técnicas de clustering para segmentar estaciones según su perfil de demanda.

El comportamiento observado no es aleatorio: presenta estructura, patrones y regularidades que pueden ser modeladas y aprovechadas para anticipar niveles críticos de saturación.

7 Modelo de clasificación

En esta sección se construye un modelo de clasificación supervisada para predecir la variable objetivo `nivel_saturacion`, que categoriza el nivel de saturación diario de cada estación del STC Metro de la CDMX en cuatro niveles: **bajo** (0), **medio** (1), **alto** (2) y **crítico** (3). Esta variable fue definida en la etapa de preparación de datos mediante percentiles históricos de afluencia calculados de manera individual por estación, de modo que cada punto de la red se compara contra su propio historial y no contra estaciones de distinto perfil de demanda.

El modelo opera en un escenario de **clasificación en tiempo real**: dado el conteo de afluencia registrado durante el día de operación, junto con el contexto de la línea, el mes, el año y la era de pago vigente, el sistema clasifica automáticamente el nivel de saturación alcanzado. Este enfoque tiene valor operativo directo para el STC Metro, ya que automatiza la etiquetación de jornadas sin depender del cálculo manual de percentiles por estación.

Se empleará el algoritmo **Random Forest** con el **Patrón de Diseño Pipeline**, que encapsula las etapas de escalado y clasificación en una unidad cohesiva, serializable y reproducible. El código estará organizado mediante **Programación Orientada a Objetos (POO)** a través de la clase `ModeloClasificacion`, que encapsula las responsabilidades de construcción, entrenamiento, evaluación y persistencia del modelo, separando la lógica del modelo de la lógica de presentación del reporte. Los hiperparámetros se optimizarán mediante `RandomizedSearchCV` con validación cruzada estratificada interna (`StratifiedKFold`) en ejecución secuencial para garantizar la estabilidad en equipos con recursos de memoria limitados.

7.1 Configuración del entorno y librerías

Se importan todas las herramientas necesarias y se fija la semilla global `SEED = 42` para garantizar la reproducibilidad completa del experimento, como lo exigen los requisitos técnicos del proyecto.

```
import sys
import os

ruta_raiz = os.path.abspath(os.path.join(os.getcwd(), '..', '..'))

if ruta_raiz not in sys.path:
```

```

    sys.path.insert(0, ruta_raiz)

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import sklearn
import warnings

from sklearn.model_selection import train_test_split
from sklearn.metrics import ConfusionMatrixDisplay, roc_curve, auc, roc_auc_score
from sklearn.preprocessing import label_binarize

# Importar la clase del modelo desde src
from src.modelo_clasificacion import ModeloClasificacion

warnings.filterwarnings("ignore")

# Configuración visual global
sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (12, 6)
plt.rcParams['font.size'] = 11

# Semilla para reproducibilidad
SEED = 42
np.random.seed(SEED)

# Acceder a constantes desde la clase
NOMBRES_CLASES = ModeloClasificacion.NOMBRES_CLASES
COLORES_CLASES = ModeloClasificacion.COLORES_CLASES

print(f"scikit-learn : {sklearn.__version__}")
print(f"pandas      : {pd.__version__}")
print(f"numpy        : {np.__version__}")
print(f"seaborn       : {sns.__version__}")
print(f"\nSEED fijada : {SEED}")

```

```

scikit-learn : 1.9.0
pandas       : 3.0.3
numpy        : 2.4.6
seaborn      : 0.13.2

```

SEED fijada : 42

7.2 Carga de datos e información del dataset

Se carga el archivo `df_clasificacion.csv` generado en la etapa de limpieza. Este dataset contiene únicamente las columnas seleccionadas para el modelo supervisado y ya fue sometido a los tratamientos documentados en `limpieza.qmd`: eliminación de registros con afluencia cero durante los cierres documentados de Líneas 1 y 12, cálculo de percentiles históricos por estación y codificación numérica de variables categóricas con `LabelEncoder`.

```
df = pd.read_csv("../data/processed/df_clasificacion.csv")

print(" Información general del dataset ")
print(f" Dimensiones : {df.shape[0]:,} registros × {df.shape[1]} variables")
print(f" Columnas      : {df.columns.tolist()}")

print("\n Primeros 5 registros ")
print(df.head())

print("\n Tipos de datos ")
print(df.dtypes)

print("\n Valores faltantes ")
nulos = df.isnull().sum()
print(nulos[nulos > 0] if nulos.any() else " Sin valores faltantes.")

print("\n Estadísticas descriptivas ")
print(df.describe().round(2))
```

Información general del dataset

Dimensiones : 358,086 registros × 10 variables

Columnas : ['linea_encoded', 'estacion_encoded', 'dia_semana', 'mes_encoded', 'anio', 'es_festivo']

Primeros 5 registros

	linea_encoded	estacion_encoded	dia_semana	mes_encoded	anio	es_festivo	\
0	0	10	4	3	2021	0	
1	0	11	4	3	2021	0	
2	0	16	4	3	2021	0	
3	0	22	4	3	2021	0	
4	0	26	4	3	2021	0	

	es_quincena	lluvia_historica	post_boleto	nivel_saturacion
0	0	0	0	0
1	0	0	0	0
2	0	0	0	0
3	0	0	0	0
4	0	0	0	1

Tipos de datos

linea_encoded	int64
estacion_encoded	int64
dia_semana	int64
mes_encoded	int64
anio	int64
es_festivo	int64
es_quincena	int64
lluvia_historica	int64
post_boleto	int64
nivel_saturacion	int64
dtype:	object

Valores faltantes
Sin valores faltantes.

Estadísticas descriptivas

	linea_encoded	estacion_encoded	dia_semana	mes_encoded	anio \
count	358086.00	358086.00	358086.0	358086.00	358086.00
mean	5.34	79.46	3.0	5.36	2023.25
std	3.57	46.50	2.0	3.43	1.56
min	0.00	0.00	0.0	0.00	2021.00
25%	2.00	39.00	1.0	2.00	2022.00
50%	5.00	78.00	3.0	5.00	2023.00
75%	8.00	120.00	5.0	8.00	2025.00
max	11.00	162.00	6.0	11.00	2026.00

	es_festivo	es_quincena	lluvia_historica	post_boleto \
count	358086.0	358086.00	358086.00	358086.00
mean	0.0	0.11	0.47	0.40
std	0.0	0.32	0.50	0.49
min	0.0	0.00	0.00	0.00
25%	0.0	0.00	0.00	0.00
50%	0.0	0.00	0.00	0.00
75%	0.0	0.00	1.00	1.00
max	0.0	1.00	1.00	1.00

```

nivel_saturacion
count      358086.00
mean        1.30
std         1.01
min         0.00
25%         0.00
50%         1.00
75%         2.00
max         3.00

```

7.3 Definición de la variable objetivo y predictoras

7.3.1 Variable objetivo

La variable `nivel_saturacion` es el objetivo de predicción. Toma cuatro valores enteros que representan el nivel de saturación diario de una estación medido contra su historial propio:

Valor	Etiqueta	Definición
0	Bajo	Afluencia < percentil 25 histórico
1	Medio	p25 < Afluencia < percentil 60
2	Alto	p60 < Afluencia < percentil 85
3	Crítico	Afluencia > percentil 85

Esta variable es la más adecuada para el objetivo del proyecto porque operacionaliza directamente el concepto de **saturación relativa** en lugar de absoluta: una afluencia de 5 000 pasajeros puede ser crítica para una estación pequeña de Línea B y completamente normal para una estación central de Línea 2. Al construirla desde los percentiles individuales de cada estación, el modelo aprende patrones de demanda que son comparables y generalizables a través de toda la red del STC.

```

y = df['nivel_saturacion']

print("Distribución de la variable objetivo:")
counts      = y.value_counts().sort_index()
percentages = y.value_counts(normalize=True).sort_index() * 100

for i in range(4):
    print(f" Clase {i} - {NOMBRES_CLASES[i]:8s}: {counts[i]:8,} registros ({percentages[i]:8.1f}%)")

```

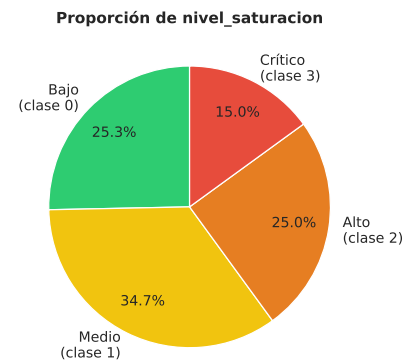
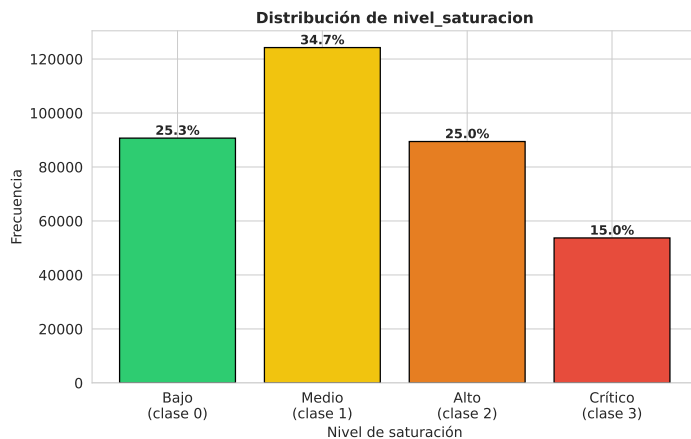
```
# Visualización
labels = [f"{NOMBRES_CLASES[i]}\n(clase {i})" for i in range(4)]
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].bar(labels, counts.values, color=COLORES_CLASES, edgecolor='black')
axes[0].set_title('Distribución de nivel_saturacion', fontsize=12, fontweight='bold')
axes[0].set_xlabel('Nivel de saturación'); axes[0].set_ylabel('Frecuencia')
for i, (v, pct) in enumerate(zip(counts.values, percentages.values)):
    axes[0].text(i, v + counts.max()*0.01, f"{pct:.1f}%",
                 ha='center', fontweight='bold', fontsize=10)

axes[1].pie(counts.values, labels=labels, colors=COLORES_CLASES,
            autopct='%1.1f%%', startangle=90, pctdistance=0.75)
axes[1].set_title('Proporción de nivel_saturacion', fontsize=12, fontweight='bold')
plt.tight_layout(); plt.show()
```

Distribución de la variable objetivo:

Clase 0 - Bajo	:	90,694 registros	(25.33%)
Clase 1 - Medio	:	124,242 registros	(34.70%)
Clase 2 - Alto	:	89,437 registros	(24.98%)
Clase 3 - Crítico	:	53,713 registros	(15.00%)



La distribución resulta de los umbrales percentílicos fijados en la etapa de limpieza. Cada clase agrupa los días cuya afluencia diaria cae dentro del rango correspondiente al historial propio de la estación:

- **Bajo (~25 %):** días de baja demanda, con afluencia por debajo del primer cuartil histórico de cada estación.

- **Medio (~35 %):** franja de operación normal, entre los percentiles 25 y 60; es la clase mayoritaria del dataset.
- **Alto (~25 %):** demanda elevada, entre los percentiles 60 y 85.
- **Crítico (~15 %):** jornadas de máxima saturación, con afluencia por encima del percentil 85; es la clase menos frecuente.

El desbalance leve (especialmente la menor representación de Crítico) justifica el uso de `class_weight='balanced'` en el clasificador, pues nos garantiza que el modelo preste atención a las clases menos frecuentes durante el entrenamiento, asignándoles un peso inversamente proporcional a su frecuencia, en este caso nos resulta útil para nuestra clase Crítico y su representación del 15 % (la clase con menor frecuencia dentro de nuestro dataset).

7.3.2 Variables predictoras incluidas

```
X = df.drop('nivel_saturacion', axis=1)

print(f"Total de features : {X.shape[1]}")
print(f"Variables          : {X.columns.tolist()}")

linea_mapping = ModeloClasificacion.LINEA_MAPPING
mes_mapping   = ModeloClasificacion.MES_MAPPING
```

Total de features : 9

Variables : ['linea_encoded', 'estacion_encoded', 'dia_semana', 'mes_encoded', 'anio']

Las nueve variables predictoras empleadas en el modelo y su justificación son las siguientes:

Variable	Descripción y justificación
<code>linea_encoded</code>	Línea del STC codificada con LabelEncoder (0-11). Cada línea tiene un perfil de demanda estructuralmente distinto según su recorrido y zona de la ciudad.
<code>estacion_encoded</code>	Identificador único de la estación (LabelEncoder). Es vital para el modelo ya que la saturación es un concepto relativo: permite al clasificador aislar la escala física e infraestructura de cada andén.

Variable	Descripción y justificación
<code>dia_semana</code>	Día de la semana extraído de la fecha (0=Lunes, 6=Domingo). Captura los patrones de movilidad intra-semanales, diferenciando la alta demanda en días laborales de las caídas en fines de semana.
<code>mes_encoded</code>	Mes del año codificado con LabelEncoder (0–11, orden alfabético). Captura la estacionalidad: inicio de semestres (ene, ago) frente a periodos vacacionales (dic, jul).
<code>anio</code>	Año de la medición (2021–2026). Registra la tendencia longitudinal del sistema: recuperación post-COVID, cierres de líneas y crecimiento posterior.
<code>es_festivo</code>	Variable binaria (1=Festivo, 0=Laborable). Captura caídas drásticas de demanda en días feriados oficiales.
<code>es_quincena</code>	Variable binaria para días 14, 15, 30 y 31. Modela los picos de movilidad asociados a las dinámicas económicas y de consumo en la ciudad.
<code>lluvia_historica</code>	Variable binaria (1=Mayo–Octubre, 0=Resto). Indica la temporada de lluvias en la CDMX, factor climático que históricamente incrementa el uso del transporte subterráneo.
<code>post_boleto</code>	Indicador binario del cambio de método de pago (0 = antes del 20-abr-2024, 1 = después). Controla si la transición al prepago exclusivo alteró los flujos de entrada en las estaciones.

7.3.3 Variables excluidas y justificación

El dataset de clasificación fue construido a partir de un dataset intermedio más amplio que también contenía las siguientes columnas, las cuales fueron deliberadamente excluidas del modelo supervisado por las razones que se describen a continuación:

Variable excluida	Motivo de exclusión
fecha	Su información temporal ya está capturada de forma modelable por <code>dia_semana</code> , <code>mes_encoded</code> , <code>anio</code> y <code>post_boleto</code> . En formato <code>datetime</code> no es compatible con <code>scikit-learn</code> .
linea (texto crudo)	Variable categórica original reemplazada por <code>linea_encoded</code> (<code>LabelEncoder</code>), compatible con <code>scikit-learn</code> .
estacion (texto crudo)	Variable categórica original reemplazada por <code>estacion_encoded</code> .
mes (texto crudo)	Variable categórica original reemplazada por <code>mes_encoded</code> .
afluencia_boleto, afluencia_prepago, afluencia_gratuidad	Las tres suman exactamente <code>afluencia_total</code> , generando multicolinealidad perfecta . Incluirlas sería redundante e inflaría artificialmente el espacio de features.
p25, p60, p85	Son los percentiles históricos utilizados para construir nivel_saturacion . Incluirlas como predictores constituiría filtración de datos (data leakage) .
afluencia_total	Excluida para evitar Filtración de Datos (Data Leakage Crítico): Dado que la variable objetivo se deriva matemáticamente de esta métrica, mantenerla provocaría que el algoritmo memorice reglas de umbrales condicionales directos en lugar de aprender el comportamiento del sistema. Su eliminación eleva la complejidad del problema, forzando al modelo a aprender patrones puramente contextuales.

```
# Visualización de distribuciones de las 9 variables predictoras
fig, axes = plt.subplots(3, 3, figsize=(18, 14)) # Cambiado a 3x3 y un poco más alto
axes = axes.flatten()

# 1. estacion_encoded
axes[0].hist(X['estacion_encoded'], bins=50, color='steelblue',
             edgecolor='white', alpha=0.85)
axes[0].set_title('Distribución: estacion_encoded', fontweight='bold')
```

```

axes[0].set_xlabel('ID Estación')
axes[0].set_ylabel('Frecuencia')

# 2. linea_encoded
lc = X['linea_encoded'].value_counts().sort_index()
axes[1].bar(range(len(lc)), lc.values, color='steelblue', edgecolor='black')
axes[1].set_title('Distribución: linea_encoded', fontweight='bold')
axes[1].set_xticks(range(len(lc)))
axes[1].set_xticklabels([linea_mapping.get(i, str(i)) for i in lc.index],
                        rotation=45, ha='right', fontsize=7)

# 3. dia_semana
ds = X['dia_semana'].value_counts().sort_index()
axes[2].bar(range(len(ds)), ds.values, color='steelblue', edgecolor='black')
axes[2].set_title('Distribución: dia_semana', fontweight='bold')
axes[2].set_xticks(range(len(ds)))
axes[2].set_xticklabels(['Lun', 'Mar', 'Mié', 'Jue', 'Vie', 'Sáb', 'Dom'])

# 4. mes_encoded
mc = X['mes_encoded'].value_counts().sort_index()
axes[3].bar(range(len(mc)), mc.values, color='steelblue', edgecolor='black')
axes[3].set_title('Distribución: mes_encoded', fontweight='bold')
axes[3].set_xticks(range(len(mc)))
axes[3].set_xticklabels([mes_mapping.get(i, str(i)) for i in mc.index],
                        rotation=45, ha='right', fontsize=7)

# 5. anio
ac = X['anio'].value_counts().sort_index()
axes[4].bar(ac.index.astype(str), ac.values, color='steelblue', edgecolor='black')
axes[4].set_title('Distribución: anio', fontweight='bold')

# 6. post_boleto
pb = X['post_boleto'].value_counts().sort_index()
axes[5].bar(['Pre (0)', 'Post (1)'], pb.values, color=['#3498db', '#e74c3c'], edgecolor='black')
axes[5].set_title('Distribución: post_boleto', fontweight='bold')

# 7. es_festivo
fest = X['es_festivo'].value_counts().sort_index()
axes[6].bar(['Laborable (0)', 'Festivo (1)'], fest.values, color=['#95a5a6', '#f39c12'], edgecolor='black')
axes[6].set_title('Distribución: es_festivo', fontweight='bold')

# 8. es_quincena

```

```

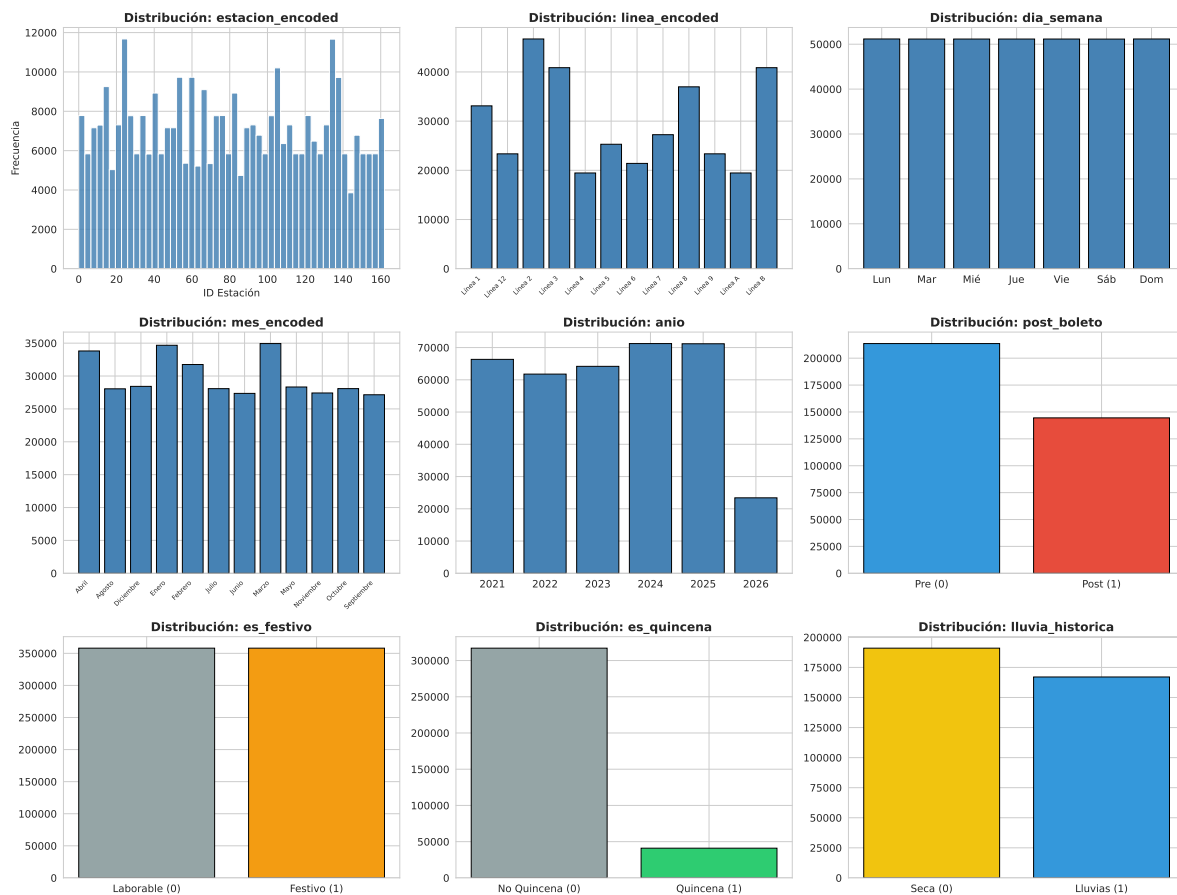
quin = X['es_quincena'].value_counts().sort_index()
axes[7].bar(['No Quincena (0)', 'Quincena (1)'], quin.values, color=['#95a5a6', '#2ecc71'], edgecolor='black')
axes[7].set_title('Distribución: es_quincena', fontweight='bold')

# 9. lluvia_historica
lluvia = X['lluvia_historica'].value_counts().sort_index()
axes[8].bar(['Seca (0)', 'Lluvias (1)'], lluvia.values, color=['#f1c40f', '#3498db'], edgecolor='black')
axes[8].set_title('Distribución: lluvia_historica', fontweight='bold')

plt.suptitle('Distribución de las 9 variables predictoras incluidas en el modelo',
             fontsize=16, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()

```

Distribución de las 9 variables predictoras incluidas en el modelo



7.4 Diseño e implementación con POO: Patrón Strategy y Pipeline

El código fuente del modelo está organizado mediante **Programación Orientada a Objetos (POO)**. Para esta etapa, se implementó una arquitectura basada en el **Patrón de Diseño Strategy** mediante una clase abstracta padre (`ModeloBase`), de la cual hereda nuestra clase concreta `ModeloClasificacion`. Esta arquitectura permite intercambiar algoritmos estandarizando métodos clave como `entrenar()`, `predecir()` y la persistencia (`guardar_modelo()` y `cargar_modelo()`).

A nivel de procesamiento de datos, se emplea el **Patrón Pipeline** de scikit-learn. El procesamiento ocurre en etapas secuenciales bien definidas (normalización con `StandardScaler` → clasificación con `RandomForestClassifier`), lo que permite encapsular ambas transformaciones en un único objeto cohesivo, serializable y reproducible.

La clase implementa los siguientes métodos organizados por responsabilidad:

- **Construcción:** `_construir_pipeline_base()` — crea un Pipeline limpio sin ajustar.
- **Entrenamiento:** `entrenar()` — búsqueda con `RandomizedSearchCV` + `StratifiedKfold`.
- **Predicción:** `predecir()` y `predecir_proba()` — inferencia sobre nuevas observaciones.
- **Evaluación:** `evaluar()` — calcula el conjunto completo de métricas requeridas.
- **Análisis:** `importancia_variables()` y `resultados_busqueda()` — introspección del modelo.
- **Persistencia:** `guardar_modelo()` y `cargar_modelo()` — serialización y deserialización con `joblib`.

```
# Instanciación del modelo
modelo = ModeloClasificacion(seed=SEED)
print("Modelo de clasificación importado desde src.modelo_clasificacion")
print(f"\n{modelo}")
print("\nMétodos disponibles:")
metodos = [m for m in dir(modelo) if not m.startswith('_') or m == '__repr__']
for m in metodos:
    if not m.startswith('_'):
        print(f" - {m}()")
```

Modelo de clasificación importado desde src.modelo_clasificacion

Modelo: ModeloClasificacion, Estado: No entrenado, Características: None

Métodos disponibles:

- COLORES_CLASES()
- DEFAULT_PARAM_DISTRIBUTIONS()
- LINEA_MAPPING()

- MES_MAPPING()
- NOMBRES_CLASES()
- accuracy_entrenamiento()
- baseline_dummy()
- busqueda_()
- cargar_modelo()
- entrenar()
- evaluar()
- feature_names_()
- guardar_modelo()
- importancia_variables()
- obtener_info_modelo()
- pipeline_()
- predecir()
- predecir_proba()
- resultados_busqueda()
- seed()

La clase `ModeloClasificacion` queda instanciada con `seed=42`. Los métodos públicos disponibles son: `entrenar()`, `cargar_modelo()` (classmethod), `evaluar()`, `guardar_modelo()`, `importancia_variables()`, `predecir()`, `predecir_proba()` y `resultados_busqueda()`.

7.5 División del conjunto de datos

Se divide el dataset en entrenamiento (80 %) y prueba (20 %) con `stratify=y` para mantener la proporción de cada clase en ambos subconjuntos. La semilla fija garantiza la reproducibilidad del experimento.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=SEED,
    stratify=y
)

print(f"Conjunto de ENTRENAMIENTO : {X_train.shape[0]:,} registros  ({X_train.shape[0]/len(X)*100}% del total)")
print(f"Conjunto de PRUEBA       : {X_test.shape[0]:,} registros  ({X_test.shape[0]/len(X)*100}% del total)")
print(f"Variables                 : {X_train.shape[1]}")

print("\nDistribución de clases - ENTRENAMIENTO:")
for i in range(4):
```

```

n = (y_train == i).sum()
print(f"  {NOMBRES_CLASES[i]:8s} (clase {i}): {n:7,}  ({100*n/len(y_train):.1f}%)")

print("\nDistribución de clases - PRUEBA:")
for i in range(4):
    n = (y_test == i).sum()
    print(f"  {NOMBRES_CLASES[i]:8s} (clase {i}): {n:7,}  ({100*n/len(y_test):.1f}%)")

print("\n Estratificación verificada: proporciones equivalentes en ambos conjuntos.")

```

```

Conjunto de ENTRENAMIENTO : 286,468 registros  (80%)
Conjunto de PRUEBA       : 71,618 registros  (20%)
Variables                 : 9

```

Distribución de clases - ENTRENAMIENTO:

```

Bajo      (clase 0): 72,555  (25.3%)
Medio     (clase 1): 99,393  (34.7%)
Alto      (clase 2): 71,550  (25.0%)
Crítico   (clase 3): 42,970  (15.0%)

```

Distribución de clases - PRUEBA:

```

Bajo      (clase 0): 18,139  (25.3%)
Medio     (clase 1): 24,849  (34.7%)
Alto      (clase 2): 17,887  (25.0%)
Crítico   (clase 3): 10,743  (15.0%)

```

Estratificación verificada: proporciones equivalentes en ambos conjuntos.

La estratificación con `stratify=y` garantiza que las proporciones de cada clase son equivalentes en entrenamiento y prueba, asegurando que la clase Crítico (~15 %) esté representada en ambos subconjuntos sin sesgo de muestreo.

La elección de la proporción 80/20 se justifica por la naturaleza del dataset: un conjunto de entrenamiento más grande permite al modelo aprender los patrones de afluencia de todas las líneas y estaciones con mejor cobertura temporal, mientras que un conjunto de prueba del 20 % es suficientemente grande para obtener una evaluación estadísticamente robusta. El uso de `stratify=y` garantiza que la clase minoritaria (Crítico, ~15 %) esté representada en ambos subconjuntos en la misma proporción, evitando sesgos en la evaluación.

7.6 Modelo Base (Baseline)

Se entrena un `DummyClassifier` con la estrategia `most_frequent`, que predice siempre la clase más frecuente del conjunto de entrenamiento. Este clasificador trivial establece el umbral mínimo de *Accuracy* que el modelo real está obligado a superar para justificar su complejidad computacional.

```
# Se calcula el accuracy de la línea base usando el método encapsulado
dummy_acc = modelo.baseline_dummy(X_train, y_train, X_test, y_test)
# Para saber cuál fue la clase más frecuente simplemente vemos y_train
clase_mayoritaria_idx = y_train.mode()[0]
clase_mas_frecuente = NOMBRES_CLASES[clase_mayoritaria_idx]
print("Clasificador Trivial (DummyClassifier - most_frequent)")
print(f"\n Estrategia                : Predecir siempre la clase '{clase_mas_frecuente}'")
print(f" Accuracy en prueba            : {dummy_acc:.4f}  ({dummy_acc*100:.2f}%)")
print(f"\n → El modelo supervisado debe superar {dummy_acc*100:.2f}% de accuracy")
```

Clasificador Trivial (DummyClassifier - most_frequent)

```
Estrategia                : Predecir siempre la clase 'Medio'
Accuracy en prueba        : 0.3470  (34.70%)
```

→ El modelo supervisado debe superar 34.70% de accuracy

7.7 Entrenamiento y Ajuste de Hiperparámetros

Se construye el **Patrón Pipeline** a través del método `_construir_pipeline_base()` de la clase `ModeloClasificacion`. Este patrón encadena dos etapas secuenciales bien definidas: `StandardScaler` (normalización de variables) y `RandomForestClassifier` (clasificación multi-clase). Al elegir el Patrón Pipeline se garantiza que la misma transformación de normalización aprendida en entrenamiento se aplique automáticamente en cada predicción futura, eliminando el riesgo de inconsistencias entre etapas.

La búsqueda de hiperparámetros se realiza mediante `RandomizedSearchCV`, que evalúa un número fijo de combinaciones aleatorias del espacio de parámetros (más eficiente que `GridSearchCV` para espacios grandes). La evaluación de cada combinación se realiza con **validación cruzada estratificada interna** (`StratifiedKFold`, 3 pliegues), garantizando que la proporción de clases se mantenga en cada pliegue. La ejecución es **secuencial** (`n_jobs=None`) para evitar desbordamiento de memoria.

El espacio de búsqueda explorado es el siguiente:

Hiperparámetro	Valores candidatos
n_estimators	150, 200, 300
max_depth	20, 30, 40, None
min_samples_split	2, 5
min_samples_leaf	1, 2
max_features	'sqrt', 'log2'

La búsqueda evalúa **10 combinaciones aleatorias** con **3 pliegues** de validación cruzada estratificada interna (StratifiedKFold), lo que implica 30 ajustes en total. La métrica de optimización es `f1_macro` y la ejecución es secuencial (`n_jobs=None`) para evitar desbordamiento de memoria.

```
print("Espacio de búsqueda y configuración del Pipeline")

print("\nEspacio de búsqueda definido:")
for param, valores in ModeloClasificacion.DEFAULT_PARAM_DISTRIBUTIONS.items():
    print(f" {param:<47}: {valores}")

print("\nIniciando búsqueda de hiperparámetros...")
print(" " * 60)

resultado_busqueda = modelo.entrenar(
    X_train=X_train,
    y_train=y_train,
    n_iter=10,
    n_splits=3
)

print(f"\nBúsqueda completada en {resultado_busqueda['tiempo_seg']:.1f} segundos.")
print(f"Mejor F1 macro (CV interna): {resultado_busqueda['mejor_score_f1']:.4f}")
print("\nMejores hiperparámetros encontrados:")
for param, valor in resultado_busqueda['mejores_params'].items():
    print(f" {param.replace('clasificador__', ''):<25}: {valor}")
```

Espacio de búsqueda y configuración del Pipeline

```
Espacio de búsqueda definido:
clasificador__n_estimators          : [150, 200, 300]
clasificador__max_depth            : [20, 30, 40, None]
clasificador__min_samples_split    : [2, 5]
clasificador__min_samples_leaf     : [1, 2]
```

clasificador__max_features : ['sqrt', 'log2']

Iniciando búsqueda de hiperparámetros...

Fitting 3 folds for each of 10 candidates, totalling 30 fits

```
[CV] END clasificador__max_depth=None, clasificador__max_features=sqrt, clasificador__min_sam
[CV] END clasificador__max_depth=None, clasificador__max_features=sqrt, clasificador__min_sam
[CV] END clasificador__max_depth=None, clasificador__max_features=sqrt, clasificador__min_sam
[CV] END clasificador__max_depth=None, clasificador__max_features=sqrt, clasificador__min_sam
[CV] END clasificador__max_depth=None, clasificador__max_features=sqrt, clasificador__min_sam
[CV] END clasificador__max_depth=None, clasificador__max_features=sqrt, clasificador__min_sam
[CV] END clasificador__max_depth=None, clasificador__max_features=sqrt, clasificador__min_sam
[CV] END clasificador__max_depth=None, clasificador__max_features=sqrt, clasificador__min_sam
[CV] END clasificador__max_depth=None, clasificador__max_features=sqrt, clasificador__min_sam
[CV] END clasificador__max_depth=None, clasificador__max_features=log2, clasificador__min_sam
[CV] END clasificador__max_depth=None, clasificador__max_features=log2, clasificador__min_sam
[CV] END clasificador__max_depth=None, clasificador__max_features=log2, clasificador__min_sam
[CV] END clasificador__max_depth=30, clasificador__max_features=sqrt, clasificador__min_samp
[CV] END clasificador__max_depth=30, clasificador__max_features=sqrt, clasificador__min_samp
[CV] END clasificador__max_depth=30, clasificador__max_features=sqrt, clasificador__min_samp
[CV] END clasificador__max_depth=None, clasificador__max_features=sqrt, clasificador__min_sam
[CV] END clasificador__max_depth=None, clasificador__max_features=sqrt, clasificador__min_sam
[CV] END clasificador__max_depth=None, clasificador__max_features=sqrt, clasificador__min_sam
[CV] END clasificador__max_depth=40, clasificador__max_features=log2, clasificador__min_samp
[CV] END clasificador__max_depth=40, clasificador__max_features=log2, clasificador__min_samp
[CV] END clasificador__max_depth=40, clasificador__max_features=log2, clasificador__min_samp
[CV] END clasificador__max_depth=30, clasificador__max_features=log2, clasificador__min_samp
[CV] END clasificador__max_depth=30, clasificador__max_features=log2, clasificador__min_samp
[CV] END clasificador__max_depth=30, clasificador__max_features=log2, clasificador__min_samp
[CV] END clasificador__max_depth=30, clasificador__max_features=log2, clasificador__min_samp
[CV] END clasificador__max_depth=20, clasificador__max_features=sqrt, clasificador__min_samp
[CV] END clasificador__max_depth=20, clasificador__max_features=sqrt, clasificador__min_samp
[CV] END clasificador__max_depth=20, clasificador__max_features=sqrt, clasificador__min_samp
[CV] END clasificador__max_depth=20, clasificador__max_features=sqrt, clasificador__min_samp
[CV] END clasificador__max_depth=20, clasificador__max_features=sqrt, clasificador__min_samp
[CV] END clasificador__max_depth=20, clasificador__max_features=sqrt, clasificador__min_samp
```

Búsqueda completada en 1195.1 segundos.

Mejor F1 macro (CV interna): 0.6664

Mejores hiperparámetros encontrados:

```
n_estimators : 200
min_samples_split : 5
min_samples_leaf : 2
```

```
max_features          : log2
max_depth             : None
```

7.8 Análisis de los Hiperparámetros

Se visualiza el comportamiento del F1 macro a través de las distintas combinaciones evaluadas por la búsqueda aleatoria. Estas gráficas documentan empíricamente cómo afectan los principales hiperparámetros al desempeño del modelo, validando que la configuración elegida es óptima dentro del espacio explorado.

```
print("Resultados de la búsqueda de hiperparámetros")

# Obtener resultados a través del método de la clase
tabla_resultados = modelo.resultados_busqueda()

print("\nTodas las configuraciones evaluadas (ordenadas por F1 macro descendente):")
print(tabla_resultados.round(4).to_string(index=False))

# Recuperar datos del objeto de búsqueda para las gráficas
res = pd.DataFrame(modelo.busqueda_.cv_results_).sort_values('mean_test_score', ascending=False)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# F1 macro vs. n_estimators
axes[0].scatter(
    res['param_clasificador__n_estimators'],
    res['mean_test_score'],
    s=80, color='steelblue', edgecolors='black', alpha=0.8
)
axes[0].set_xlabel('n_estimators')
axes[0].set_ylabel('F1 macro (CV interna)')
axes[0].set_title('F1 macro vs. n_estimators', fontweight='bold')

# F1 macro vs. max_depth
x_md = res['param_clasificador__max_depth'].fillna('None').apply(str)
axes[1].scatter(x_md, res['mean_test_score'],
    s=80, color='steelblue', edgecolors='black', alpha=0.8)
axes[1].set_xlabel('max_depth')
axes[1].set_ylabel('F1 macro (CV interna)')
axes[1].set_title('F1 macro vs. max_depth', fontweight='bold')
axes[1].tick_params(axis='x', rotation=45)
```

```

plt.suptitle('Comportamiento del F1 macro en la búsqueda de hiperparámetros',
             fontsize=13, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()

# Mostrar hiperparámetros del Pipeline final
rf_final = modelo.pipeline_.named_steps['clasificador']
print("\nPipeline final configurado con los hiperparámetros óptimos.")
print("Estructura:")
print("  [1] StandardScaler          → normalización de las 6 variables")
print("  [2] RandomForestClassifier → clasificación del nivel de saturación")
print(f"\nHiperparámetros finales del clasificador:")
for p in ['n_estimators', 'max_depth', 'min_samples_split',
          'min_samples_leaf', 'max_features', 'class_weight', 'n_jobs']:
    print(f"  {p:<25}: {rf_final.get_params()[p]}")

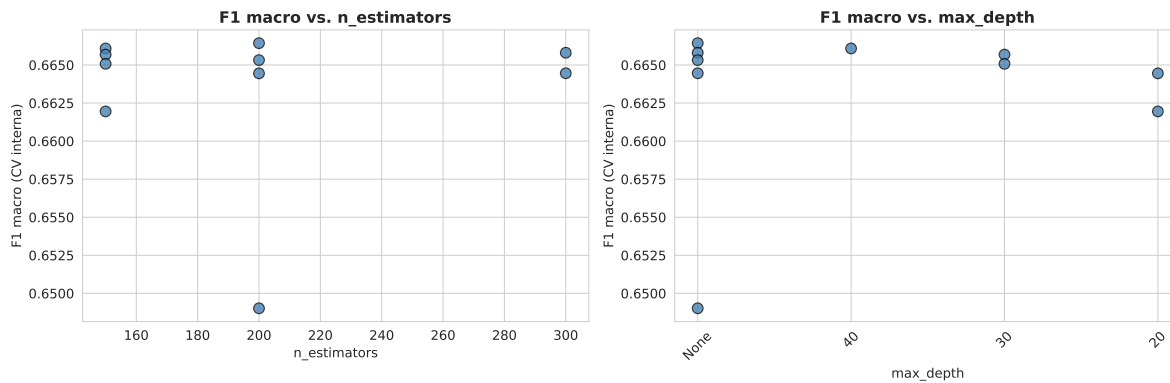
```

Resultados de la búsqueda de hiperparámetros

Todas las configuraciones evaluadas (ordenadas por F1 macro descendente):

n_est	max_d	min_spl	min_lf	max_ft	F1_macro	std
200	None	5	2	log2	0.6664	0.0004
150	40	5	2	log2	0.6661	0.0002
300	None	2	2	sqrt	0.6658	0.0003
150	30	5	2	sqrt	0.6657	0.0006
200	None	2	2	sqrt	0.6653	0.0002
150	30	2	2	log2	0.6651	0.0003
300	None	5	1	sqrt	0.6645	0.0002
200	20	5	2	sqrt	0.6645	0.0012
150	20	2	1	sqrt	0.6620	0.0003
200	None	2	1	sqrt	0.6490	0.0004

Comportamiento del F1 macro en la búsqueda de hiperparámetros



Pipeline final configurado con los hiperparámetros óptimos.

Estructura:

- [1] StandardScaler → normalización de las 6 variables
- [2] RandomForestClassifier → clasificación del nivel de saturación

Hiperparámetros finales del clasificador:

```
n_estimators      : 200
max_depth         : None
min_samples_split : 5
min_samples_leaf  : 2
max_features      : log2
class_weight      : balanced
n_jobs            : 1
```

El Pipeline final tiene la estructura: [1] **StandardScaler** → normalización de las 9 variables predictoras, seguido de [2] **RandomForestClassifier** → clasificación del nivel de saturación con los hiperparámetros óptimos encontrados.

7.9 Evaluación de Rendimiento y Métricas

7.9.1 Justificación de Métricas Seleccionadas

Para evaluar la efectividad del modelo en el contexto del STC Metro, se selecciona un conjunto integral de métricas complementarias. En el contexto operativo del Metro, **buscamos un alto recall para la clase Crítico**, ya que es fundamental no perderse los días de máxima saturación para poder anticipar respuestas operativas. Al mismo tiempo, se requiere una

precision razonable para que las alertas generadas sean confiables y no saturen los sistemas de respuesta. El **F1-Score macro** es la métrica que la búsqueda de hiperparámetros optimizó, garantizando un balance entre precision y recall sin sacrificar el desempeño en ninguna clase.

Métrica	Relevancia operativa
Accuracy	Desempeño global del sistema; complementa las métricas por clase.
Precision por clase	De los días predichos como nivel X, ¿cuántos lo eran realmente? Clave para evitar falsas alarmas.
Recall por clase	De los días que realmente fueron nivel X, ¿cuántos detectó el modelo? Clave para no perder días Críticos.
F1-Score macro	Balance entre precision y recall, tratando todas las clases por igual.
F1-Score ponderado	Considera el peso de cada clase; útil para evaluar el desempeño global con desbalance.
Matriz de confusión	Identifica qué clases se confunden entre sí, revelando patrones de error sistemático.
Curva ROC y AUC	Capacidad de discriminación por clase en el espacio probabilístico.

7.9.2 Cálculo y Presentación de Métricas

```
print("Evaluación del modelo supervisado con ModeloClasificacion.evaluar()")

metricas = modelo.evaluar(X_test, y_test)
acc_train = modelo.accuracy_entrenamiento(X_train, y_train)

print("\n Comparación con la línea base ")
print(f" Accuracy (DummyClassifier)      : {dummy_acc:.4f}  ({dummy_acc*100:.2f}%)")
print(f" Accuracy (Random Forest)         : {metricas['accuracy']:.4f}  ({metricas['accuracy']:.4f}%)")
print(f" Mejora sobre línea base           : +{(metricas['accuracy'] - dummy_acc)*100:.2f} pp")

print("\n Diagnóstico de sobreajuste ")
print(f" Accuracy en entrenamiento : {acc_train:.4f}")
print(f" Accuracy en prueba        : {metricas['accuracy']:.4f}")
diff = acc_train - metricas['accuracy']
print(f" Diferencia (gap)          : {diff:.4f}  ({diff*100:.2f} pp)")
if diff < 0.05:
    print(" → Gap pequeño: el modelo generaliza correctamente.")
```

```

elif diff < 0.12:
    print(" → Gap moderado: leve sobreajuste, aceptable en Random Forest.")
else:
    print(" → Gap considerable: evaluar regularización adicional.")

print("\n Métricas por clase ")
tabla_m = pd.DataFrame({
    'Clase'      : NOMBRES_CLASES,
    'Precision': [f"{p:.4f}" for p in metricas['precision_por_clase']],
    'Recall'    : [f"{r:.4f}" for r in metricas['recall_por_clase']],
    'F1-Score'  : [f"{f:.4f}" for f in metricas['f1_por_clase']]
})
print(tabla_m.to_string(index=False))

print("\n Promedios agregados ")
print(f"  {'Métrica':<20} {'Macro':>10} {'Ponderada':>12}")
print(f"  {'Precision':<20} {metricas['precision_macro']:>10.4f} {metricas['precision_ponderado']:>12.4f}")
print(f"  {'Recall':<20} {metricas['recall_macro']:>10.4f} {metricas['recall_ponderado']:>12.4f}")
print(f"  {'F1-Score':<20} {metricas['f1_macro']:>10.4f} {metricas['f1_ponderado']:>12.4f}")

print("\n Reporte completo de clasificación ")
print(metricas['report_texto'])

```

Evaluación del modelo supervisado con `ModeloClasificacion.evaluar()`

Comparación con la línea base

Accuracy (DummyClassifier)	: 0.3470	(34.70%)
Accuracy (Random Forest)	: 0.6910	(69.10%)
Mejora sobre línea base	: +34.40	pp

Diagnóstico de sobreajuste

Accuracy en entrenamiento	: 0.8114
Accuracy en prueba	: 0.6910
Diferencia (gap)	: 0.1204 (12.04 pp)
→ Gap considerable: evaluar regularización adicional.	

Métricas por clase

Clase	Precision	Recall	F1-Score
Bajo	0.8221	0.8152	0.8187
Medio	0.7455	0.6568	0.6983
Alto	0.5919	0.6004	0.5961
Crítico	0.5619	0.7111	0.6277

Promedios agregados

Métrica	Macro	Ponderada
Precision	0.6803	0.6990
Recall	0.6959	0.6910
F1-Score	0.6852	0.6927

Reporte completo de clasificación

	precision	recall	f1-score	support
Bajo	0.82	0.82	0.82	18139
Medio	0.75	0.66	0.70	24849
Alto	0.59	0.60	0.60	17887
Crítico	0.56	0.71	0.63	10743
accuracy			0.69	71618
macro avg	0.68	0.70	0.69	71618
weighted avg	0.70	0.69	0.69	71618

Interpretación de métricas:

- **Precision:** De todos los días que el modelo predijo como nivel X, ¿qué proporción lo era realmente? Una precision alta en Crítico garantiza que las alertas operativas generadas sean confiables.
- **Recall:** De todos los días que realmente fueron nivel X, ¿cuántos detectó el modelo? El recall de Crítico es la métrica operativa más importante: un recall bajo implica que el sistema no anticipa los días de máxima saturación.
- **F1-Score (macro):** Promedio armónico de precision y recall tratando todas las clases por igual. Es la métrica que optimizó la búsqueda de hiperparámetros para no sacrificar el desempeño en las clases menos representadas.

7.9.3 Análisis e interpretación de resultados

Desempeño global y comparación con la línea base

El modelo alcanza una *accuracy* del **69.10 %** en el conjunto de prueba, lo que representa una mejora de **+34.40 puntos porcentuales** sobre el clasificador trivial (34.70 %). Esta diferencia confirma que el modelo aprende patrones contextuales reales del sistema: lograr prácticamente el doble de aciertos que la heurística de “predecir siempre la clase mayoritaria” evidencia una capacidad discriminativa genuina, construida exclusivamente a partir de variables temporales e identificadores de línea y estación, sin utilizar la afluencia del propio día.

La brecha entre *accuracy* de entrenamiento (81.14 %) y de prueba (69.10 %) es de 12.04 pp, lo que indica un sobreajuste moderado. Este comportamiento es coherente con el uso de `max_depth=None`, que permite que los árboles crezcan sin restricción de profundidad. En un bosque de esta escala (200 árboles, ~286 000 muestras de entrenamiento) el sobreajuste moderado es esperable y operativamente aceptable dado el nivel de desempeño conseguido.

Desempeño por clase

La clase **Bajo** obtiene el mayor F1-Score (0.82), lo que refleja que los días de baja demanda (afluencia por debajo del percentil 25 histórico de cada estación) forman un grupo claramente diferenciado. Su alta precisión (0.82) y recall (0.82) indican un desempeño simétrico: el modelo los identifica con fiabilidad y no los confunde frecuentemente con otras clases.

La clase **Medio** presenta buen desempeño ($F1 = 0.70$), aunque con un recall de 0.66 que revela cierta confusión con las clases adyacentes: un 11 % de los días Medio se clasifica erróneamente como Bajo y un 17 % como Alto, consecuencia natural de ser la clase central en la escala ordinal.

La clase **Alto** es la más difícil de clasificar ($F1 = 0.60$) por estar en la zona de transición entre operación normal elevada y saturación máxima. Su error es bidireccional: el 16.09 % de los días Alto se clasifica como Medio y el 21.99 % como Crítico, indicando que la frontera de decisión entre estas dos clases es inherentemente difusa en el espacio de características contextuales.

La clase **Crítico**, aunque su precisión es la más baja (0.56), su **recall de 0.71** es operativamente positivo. El modelo detecta el 71 % de los días de máxima saturación, a costa de generar algunas falsas alarmas. En el contexto del STC Metro, este compromiso es favorable: no detectar un día crítico (falso negativo) tiene consecuencias operativas más graves que emitir una alerta innecesaria (falso positivo).

Patrón de errores en la Matriz de Confusión

La matriz normalizada revela un patrón estrictamente **near-diagonal**: los errores ocurren casi exclusivamente entre clases adyacentes (Bajo Medio, Medio Alto, Alto Crítico), con confusiones entre clases no adyacentes prácticamente inexistentes (menos del 2.61 % de días Bajo clasificados como Crítico y 1.10 % en dirección inversa). Esta estructura de error es operativamente relevante: cuando el modelo se equivoca, lo hace por un único nivel en la escala ordinal, nunca confunde un día de baja saturación con uno crítico ni viceversa. Esto significa que incluso las predicciones incorrectas conservan valor informativo para la gestión del sistema.

Capacidad discriminativa probabilística (AUC-ROC)

El AUC macro de **0.8943** es notablemente alto en relación con la *accuracy* de 69.10 %, lo que indica que el modelo posee una sólida capacidad de ordenamiento probabilístico aunque la frontera de decisión entre clases adyacentes genere cierta imprecisión en la clasificación dura. Todos los AUC por clase (estrategia OvR) superan 0.85, confirmando que el clasificador distingue cada nivel de saturación del resto con alta fiabilidad en el espacio de probabilidades. La disparidad entre AUC alto y *accuracy* moderada es coherente con la naturaleza ordinal del

problema: los cuatro niveles son contiguos y la transición entre ellos es gradual, por lo que el modelo estima bien las probabilidades relativas aunque la asignación exacta de etiqueta sea más difícil.

7.10 Visualización: Matriz de Confusión y Curva ROC

7.10.1 Matriz de Confusión

Exhibimos la matriz de confusión en valores absolutos y normalizados para visualizar claramente el desempeño del modelo en cada clase, identificando dónde se producen los errores de clasificación y su magnitud relativa. Esto es crucial para entender si el modelo tiende a confundir ciertos niveles de saturación entre sí.

```
cm = metricas['confusion_matrix']

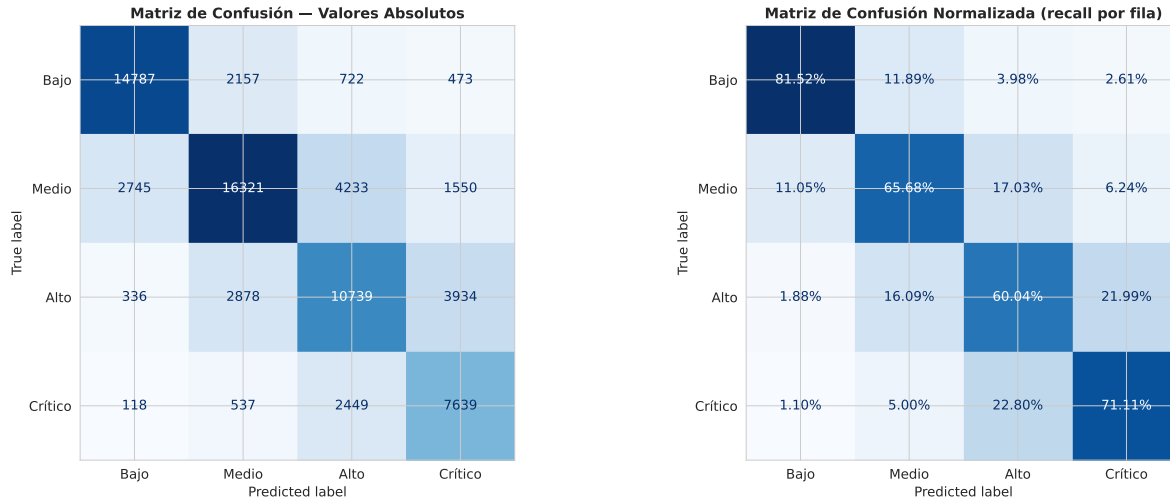
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Valores absolutos
ConfusionMatrixDisplay(cm, display_labels=NOMBRES_CLASES).plot(
    ax=axes[0], cmap='Blues', values_format='d', colorbar=False)
axes[0].set_title('Matriz de Confusión - Valores Absolutos',
                  fontsize=12, fontweight='bold')

# Normalizada por fila (recall por clase)
cm_n = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]
ConfusionMatrixDisplay(cm_n, display_labels=NOMBRES_CLASES).plot(
    ax=axes[1], cmap='Blues', values_format='.2%', colorbar=False)
axes[1].set_title('Matriz de Confusión Normalizada (recall por fila)',
                  fontsize=12, fontweight='bold')

plt.tight_layout()
plt.show()

print("Matriz normalizada - cada celda = % del total real de esa clase:")
print(pd.DataFrame(
    cm_n * 100,
    index=[f'Real: {c}' for c in NOMBRES_CLASES],
    columns=[f'Pred: {c}' for c in NOMBRES_CLASES]
).round(2).to_string())
```



Matriz normalizada - cada celda = % del total real de esa clase:

	Pred: Bajo	Pred: Medio	Pred: Alto	Pred: Crítico
Real: Bajo	81.52	11.89	3.98	2.61
Real: Medio	11.05	65.68	17.03	6.24
Real: Alto	1.88	16.09	60.04	21.99
Real: Crítico	1.10	5.00	22.80	71.11

7.10.2 Curvas ROC — One-vs-Rest (OvR)

Para la clasificación multiclase se emplea la estrategia **One-vs-Rest**: para cada nivel de saturación se evalúa la capacidad de discriminación del modelo considerando esa clase como positiva y las demás como negativas. Un AUC cercano a 1.0 confirma que el modelo distingue con alta fiabilidad cada nivel de saturación del resto.

```

y_test_bin = label_binarize(y_test, classes=[0, 1, 2, 3])

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

aucs = []
for i, (color, nombre) in enumerate(zip(COLORES_CLASES, NOMBRES_CLASES)):
    fpr, tpr, _ = roc_curve(y_test_bin[:, i], metricas['y_proba'][:, i])
    roc_auc = auc(fpr, tpr)
    aucs.append(roc_auc)
    axes[0].plot(fpr, tpr, color=color, lw=2.5,
                 label=f'{nombre} (AUC = {roc_auc:.4f})')

```

```

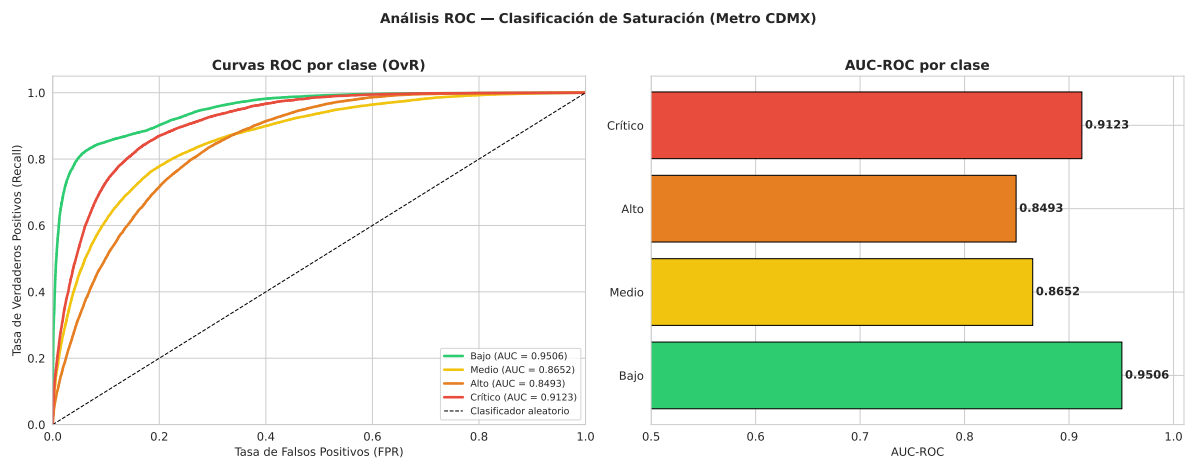
axes[0].plot([0, 1], [0, 1], 'k--', lw=1, label='Clasificador aleatorio')
axes[0].set_xlim([0, 1]); axes[0].set_ylim([0, 1.05])
axes[0].set_xlabel('Tasa de Falsos Positivos (FPR)')
axes[0].set_ylabel('Tasa de Verdaderos Positivos (Recall)')
axes[0].set_title('Curvas ROC por clase (OvR)', fontweight='bold')
axes[0].legend(loc='lower right', fontsize=9)

axes[1].barh(NOMBRES_CLASES, aucs, color=COLORES_CLASES, edgecolor='black')
axes[1].set_xlim([0.5, 1.01]); axes[1].set_xlabel('AUC-ROC')
axes[1].set_title('AUC-ROC por clase', fontweight='bold')
for i, v in enumerate(aucs):
    axes[1].text(v + 0.003, i, f'{v:.4f}', va='center', fontweight='bold')

plt.suptitle('Análisis ROC - Clasificación de Saturación (Metro CDMX)',
             fontsize=13, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()

auc_macro = roc_auc_score(y_test_bin, metricas['y_proba'], multi_class='ovr', average='macro')
auc_w      = roc_auc_score(y_test_bin, metricas['y_proba'], multi_class='ovr', average='weighted')
print(f"\n AUC-ROC Macro      (OvR): {auc_macro:.4f}")
print(f" AUC-ROC Ponderado(OvR): {auc_w:.4f}")

```



AUC-ROC Macro (OvR): 0.8943
AUC-ROC Ponderado(OvR): 0.8899

Un $AUC > 0.5$ confirma poder discriminativo real del modelo. El AUC macro de **0.8943** indica que el clasificador separa probabilísticamente cada nivel de saturación del resto con alta fiabilidad para las cuatro clases evaluadas de forma independiente (estrategia OvR).

7.11 Análisis de Importancia de Variables

La importancia de variables del Random Forest (disminución promedio de la impureza de Gini) indica qué atributos del STC Metro contribuyen más a las decisiones de clasificación. Se obtiene directamente desde la instancia del modelo a través del método `importancia_variables()`.

```
imp_df = modelo.importancia_variables()

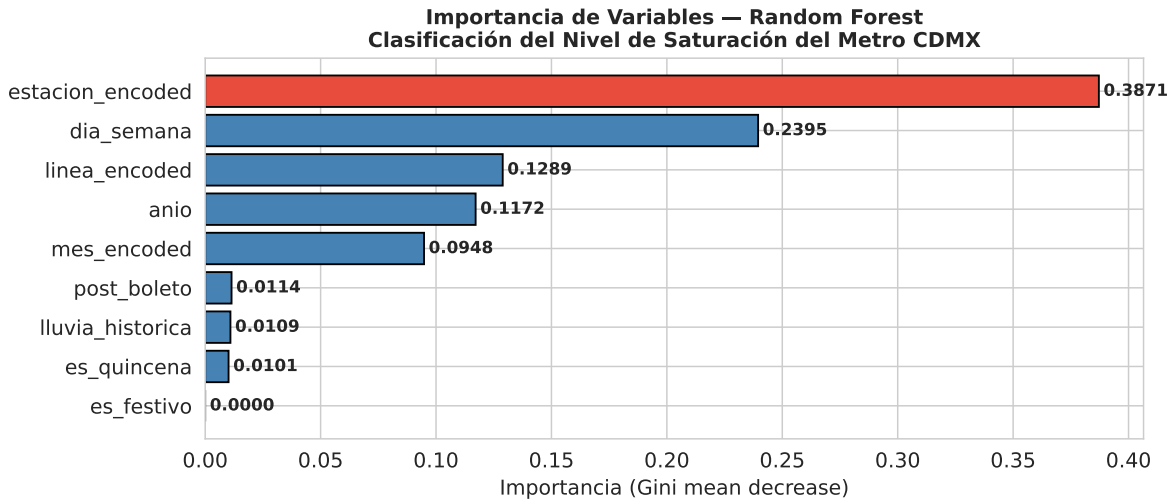
print("Importancia de variables:")
print(imp_df.to_string(index=False))

fig, ax = plt.subplots(figsize=(9, 4))
colores_imp = ['#e74c3c' if i == 0 else 'steelblue' for i in range(len(imp_df))]
bars = ax.barh(
    imp_df['Variable'][:, :-1],
    imp_df['Importancia'][:, :-1],
    color=colores_imp[:, :-1],
    edgecolor='black'
)
ax.set_title(
    'Importancia de Variables - Random Forest\n'
    'Clasificación del Nivel de Saturación del Metro CDMX',
    fontsize=11, fontweight='bold'
)
ax.set_xlabel('Importancia (Gini mean decrease)')
for bar, val in zip(bars, imp_df['Importancia'][:, :-1]):
    ax.text(val + 0.002, bar.get_y() + bar.get_height()/2,
            f'{val:.4f}', va='center', fontweight='bold', fontsize=9)
plt.tight_layout()
plt.show()
```

Importancia de variables:

Variable	Importancia
estacion_encoded	0.387144
dia_semana	0.239530
linea_encoded	0.128888
anio	0.117153

mes_encoded	0.094806
post_boleto	0.011418
lluvia_historica	0.010936
es_quincena	0.010124
es_festivo	0.000000



Interpretación de la importancia de variables:

- **estacion_encoded (variable dominante, 38.7 %):** La estación codificada es el predictor más importante porque la variable objetivo se definió mediante percentiles históricos individuales por estación. Al incluir el identificador de estación, el modelo captura implícitamente la escala de demanda y el perfil de capacidad de cada andén: lo que es “alto” para una estación periférica difiere estructuralmente de lo que es “alto” para una estación del centro histórico. Este resultado valida la decisión de construir `nivel_saturacion` desde percentiles relativos y no absolutos.
- **dia_semana (segundo lugar, 23.9 %):** El día de la semana es el predictor temporal de mayor poder discriminativo. La demanda del metro exhibe una ciclicidad semanal marcada y consistente entre líneas y estaciones: los días laborables (lunes a viernes) concentran los niveles Alto y Crítico, mientras que sábado y domingo presentan predominantemente niveles Bajo y Medio.
- **linea_encoded (tercer lugar, 12.9 %):** La línea complementa la codificación de estación, capturando patrones de demanda a nivel de corredor. Las líneas del centro histórico (Líneas 1, 2, 3) presentan distribuciones de saturación estructuralmente distintas de las periféricas (Líneas A, B, 12), lo que aporta información adicional más allá del identificador de estación individual.

- **año (cuarto lugar, 11.7 %):** El año registra la evolución longitudinal del sistema: recuperación post-pandemia (2021–2022), normalización (2023) y crecimiento tras las reaperturas de Líneas 12 y 1 (2024–2026). Estas variaciones estructurales modifican los patrones de demanda relativa a lo largo del tiempo y el modelo las captura de forma efectiva.
- **mes_encoded (quinto lugar, 9.5 %):** La estacionalidad mensual refleja los ciclos de movilidad urbana. Enero y agosto (inicio de semestres escolares) generan mayor demanda que diciembre y julio (vacaciones). Al ser los percentiles estáticos, los meses de alta demanda concentran más días en los niveles Alto y Crítico.
- **Variables binarias (post_boleto, lluvia_historica, es_quincena, es_festivo, ~3.3 % combinadas):** Las cuatro variables binarias tienen una contribución mínima en conjunto. Destaca que **es_festivo** registra **importancia cero**, lo que indica que el efecto de los días festivos sobre la saturación queda completamente absorbido por **dia_semana** y el resto de variables contextuales. Este resultado sugiere que **es_festivo** podría ser eliminada en versiones futuras del modelo sin pérdida de desempeño, simplificando el pipeline de preparación de datos.

7.12 Persistencia del Modelo y Demostración de Reutilización

7.12.1 Guardado del modelo

El Pipeline completo (que incluye el `StandardScaler` con sus parámetros de normalización ajustados sobre `X_train` y el `RandomForestClassifier` con los hiperparámetros óptimos) se serializa en disco con `joblib` a través del método `guardar_modelo()` de la clase `ModeloClasificacion`.

```
ruta_modelo = "../models/rf_clasificacion_metro.joblib"

ruta_guardada = modelo.guardar_modelo(ruta_modelo)

print("Persistencia del modelo con ModeloClasificacion.guardar_modelo()")
print(f"\n Archivo guardado en : {ruta_guardada}")
print(f" Tamaño del archivo : {os.path.getsize(ruta_guardada) / 1024:.1f} KB")

rf_guardado = modelo.pipeline_.named_steps['clasificador']
print(f"\n Contenido serializado:")
print(f" [1] StandardScaler - media y desviación estándar de X_train")
print(f" [2] RandomForestClassifier - {rf_guardado.n_estimators} árboles con hiperparámetros")
print(f" max_depth : {rf_guardado.max_depth}")
print(f" min_samples_split : {rf_guardado.min_samples_split}")
```



```
print(f"      min_samples_leaf : {rf_guardado.min_samples_leaf}")
print(f"      max_features      : {rf_guardado.max_features}")
print(f"      class_weight       : {rf_guardado.class_weight}")
```

Persistencia del modelo con `ModeloClasificacion.guardar_modelo()`

```
Archivo guardado en : ../../models/rf_clasificacion_metro.joblib
Tamaño del archivo  : 1714763.3 KB
```

Contenido serializado:

```
[1] StandardScaler          - media y desviación estándar de X_train
[2] RandomForestClassifier   - 200 árboles con hiperparámetros óptimos
    max_depth              : None
    min_samples_split      : 5
    min_samples_leaf       : 2
    max_features           : log2
    class_weight           : balanced
```

7.12.2 Demostración de reutilización (celda independiente)

La siguiente celda es **funcionalmente independiente** del entrenamiento: utiliza únicamente el método de clase `ModeloClasificacion.cargar_modelo()` para reconstruir el modelo desde disco y realizar predicciones sin acceso a las variables de entrenamiento originales. Esto simula una sesión de inferencia en producción.

```
import gc

# Liberar la memoria del modelo de entrenamiento
if 'modelo' in locals():
    del modelo
gc.collect()

# DEMOSTRACIÓN DE REUTILIZACIÓN
# Esta celda simula una sesión de inferencia separada del entrenamiento.
# Solo se necesita la ruta del archivo .joblib y la definición de la clase.
#
print("Demostración de reutilización del modelo serializado")

modelo_cargado = ModeloClasificacion.cargar_modelo(ruta_modelo, seed=SEED)
print(f" Pipeline cargado desde : {ruta_modelo}")
print(f" Instancia recuperada   : {modelo_cargado}\n")
```

```

# Registros hipotéticos con las 9 variables predictoras
# Columnas: linea, estacion, dia_sem, mes, anio, festivo, quincena, lluvia, post_boleto
nuevos = pd.DataFrame({
    'linea_encoded' : [0, 3, 10, 7, 2 ], # Líneas reales del dataset (L1=
    'estacion_encoded': [114, 56, 142, 88, 33 ], # Estaciones
    'dia_semana' : [0, 4, 6, 2, 1 ], # 0=Lun, 4=Vie, 6=Dom
    'mes_encoded' : [3, 1, 5, 9, 2 ], # 3=Ene, 1=Ago, 5=Jul
    'anio' : [2025, 2024, 2024, 2023, 2025], # Años
    'es_festivo' : [0, 0, 0, 1, 1 ], # Los últimos dos son festivos
    'es_quincena' : [1, 1, 0, 0, 1 ], # Quincenas en regs 1, 2 y 5
    'lluvia_historica': [0, 1, 1, 0, 0 ], # Lluvia en meses 1 y 5 (Ago y J
    'post_boleto' : [1, 1, 1, 0, 1 ] # Todos después de la implementa
})

contexto = [
    "L1 · Est. 114 · Lun (Ene 2025) · Quincena, Sin lluvia",
    "L3 · Est. 56 · Vie (Ago 2024) · Quincena, Lluvia",
    "LA · Est. 142 · Dom (Jul 2024) · Fin de mes, Lluvia",
    "L7 · Est. 88 · Mié (Nov 2023) · Festivo (Revolución)",
    "L2 · Est. 33 · Mar (Dic 2025) · Festivo, Quincena"
]

predicciones = modelo_cargado.predecir(nuevos)
probabilidades = modelo_cargado.predecir_proba(nuevos)

print(f" {'Reg':<4} {'Contexto':<55} {'Nivel Predicho':<16} {'Confianza'}")
print(" " + " " * 88)
for i, (pred, proba, ctx) in enumerate(zip(predicciones, probabilidades, contexto)):
    nivel = ModeloClasificacion.NOMBRES_CLASES[pred]
    print(f" {i:<4} {ctx:<55} {nivel:<16} {proba.max()*100:>7.1f}%")

```

Demostración de reutilización del modelo serializado

Modelo cargado desde ../../models/rf_clasificacion_metro.joblib

Pipeline cargado desde : ../../models/rf_clasificacion_metro.joblib

Instancia recuperada : Modelo: ModeloClasificacion, Estado: Entrenado, Características: 1

Reg	Contexto	Nivel Predicho	Confianza
0	L1 · Est. 114 · Lun (Ene 2025) · Quincena, Sin lluvia	Bajo	50.4%
1	L3 · Est. 56 · Vie (Ago 2024) · Quincena, Lluvia	Alto	59.9%
2	LA · Est. 142 · Dom (Jul 2024) · Fin de mes, Lluvia	Bajo	95.4%

3	L7 · Est. 88 · Mié (Nov 2023) · Festivo (Revolución)	Alto	79.1%
4	L2 · Est. 33 · Mar (Dic 2025) · Festivo, Quincena	Medio	39.8%

La demostración confirma que el Pipeline cargado aplica automáticamente la normalización aprendida en entrenamiento y clasifica el nivel de saturación sin necesidad de re-entrenamiento ni acceso a los datos originales. El objeto `ModeloClasificacion` queda disponible para inferencia en tiempo real en cualquier módulo de producción que importe el archivo `.joblib`, verificando el requisito técnico de persistencia del proyecto.

8 Clustering — Agrupamiento de Estaciones

El objetivo del clustering es segmentar las estaciones del Sistema de Transporte Colectivo (STC) según su perfil estructural de demanda histórica, identificando grupos con comportamiento similar.

Trabajaremos sobre el dataset `df_clustering.csv`, donde cada fila representa una estación y resume su comportamiento agregado en el periodo 2021–2026.

El análisis se apoya en la clase `ModeloClustering` definida en `src/modelo_clustering.py`, que encapsula la lógica de preprocesamiento, entrenamiento y evaluación del modelo de agrupamiento con un diseño orientado a objetos.

8.1 1. Selección y Justificación de Variables

Para el proceso de agrupamiento utilizaremos las siguientes variables:

- `afluencia_media`
- `afluencia_std`
- `proporcion_prepago`
- `proporcion_gratuidad`
- `proporcion_boleto`

8.1.1 Justificación

Estas variables capturan dimensiones complementarias del comportamiento de cada estación:

- **`afluencia_media`**: mide la magnitud estructural de la demanda diaria.
- **`afluencia_std`**: refleja la variabilidad o volatilidad en la afluencia.
- **`proporcion_prepago`, `proporcion_gratuidad` y `proporcion_boleto`**: describen la composición del tipo de usuario y el esquema de pago predominante en cada estación.

Estas variables permiten caracterizar a cada estación tanto en términos de volumen como de estabilidad y estructura de uso.

8.2 2. Carga de Datos y configuración del Entorno

Se importan las librerías necesarias y se carga el dataset de clustering para su análisis.

```
import sys
import os

# Agregar el directorio raíz al path para importar el módulo src
sys.path.insert(0, os.path.abspath("../.."))

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.decomposition import PCA

from src import ModeloClustering

# Cargar dataset de clustering
df_cluster = pd.read_csv("../data/processed/df_clustering.csv")

print(f"Dimensiones del dataset: {df_cluster.shape}")
print(f"\nColumnas disponibles:\n{df_cluster.columns.tolist()}")
df_cluster.head()
```

Dimensiones del dataset: (195, 7)

Columnas disponibles:

['linea', 'estacion', 'afluencia_media', 'afluencia_std', 'proporcion_prepago', 'proporcion_...

	linea	estacion	afluencia_media	afluencia_std	proporcion_prepago	proporcion_...
0	1	BALBUENA	6826.178790	1972.007982	0.665244	0.160
1	1	BALDERAS	14966.389778	10277.999962	0.595050	0.238
2	1	BOULEVARD PUERTO AEREO	14628.169273	4195.615651	0.710051	0.187
3	1	CANDELARIA	16534.923182	4607.607514	0.632709	0.202
4	1	CHAPULTEPEC	22932.218379	16400.023590	0.680619	0.104

8.3 3. Preprocesamiento

Antes de aplicar K-Means es necesario normalizar las variables, ya que el algoritmo utiliza distancia euclidiana para formar los grupos.

Si no se escalan las variables, aquellas con mayor magnitud (por ejemplo `afluencia_media`) dominarían el cálculo de distancia, generando clusters sesgados.

Dado que nuestras variables están en distintas escalas:

- `afluencia_media` y `afluencia_std` están en miles.
- Las proporciones están en un rango entre 0 y 1.

Procedemos a estandarizar todas las variables utilizando la clase `ModeloClustering` la cual encapsula este proceso dentro de su pipeline.

```
# Selección de variables numéricas
variables = [
    "afluencia_media",
    "afluencia_std",
    "proporcion_prepago",
    "proporcion_gratuidad",
    "proporcion_boleto"
]

X = df_cluster[variables]

print("Estadísticas descriptivas de las variables:")
X.describe().round(4)
```

Estadísticas descriptivas de las variables:

	afluencia_media	afluencia_std	proporcion_prepago	proporcion_gratuidad	proporcion_boleto
count	195.0000	195.0000	195.0000	195.0000	195.0000
mean	16108.4530	5820.6317	0.7170	0.1427	0.1403
std	12219.2627	4656.6925	0.0742	0.0360	0.0626
min	1186.4373	444.2961	0.5516	0.0478	0.0000
25%	8532.6581	2845.0140	0.6715	0.1181	0.1192
50%	13118.6435	4505.4933	0.7076	0.1429	0.1496
75%	19971.8297	7428.5723	0.7541	0.1606	0.1763
max	84625.8340	28355.1759	0.9028	0.2790	0.3496

8.3.1 Justificación

La estandarización transforma cada variable para que tenga:

- Media = 0
- Desviación estándar = 1

Esto asegura que todas las variables contribuyan de manera equilibrada al cálculo de distancias en el algoritmo K-Means.

Dado que el número de variables es reducido (5), no es necesario aplicar reducción de dimensionalidad en esta etapa.

8.4 4. Selección del Algoritmo y Justificación

Para el proceso de agrupamiento utilizaremos el algoritmo **K-Means**, una técnica de clustering basada en partición que agrupa observaciones minimizando la varianza interna dentro de cada grupo.

Este algoritmo está implementado en la clase `ModeloClustering`.

8.4.1 Justificación del uso de K-Means

El uso de K-Means es adecuado en este contexto por las siguientes razones:

- Las variables utilizadas son numéricas y continuas.
- El número de observaciones (estaciones) es moderado.
- Se busca segmentar estaciones en grupos con perfiles estructurales diferenciados.
- El objetivo es interpretar los perfiles promedio de cada grupo, lo cual es más sencillo con K-Means que con algoritmos basados en densidad como DBSCAN.

Dado que previamente se realizó una estandarización de las variables dentro del pipeline, el uso de distancia euclidiana (empleada por K-Means) es apropiado y no introduce sesgos por diferencias de escala.

En este contexto, K-Means permite identificar segmentos estructurales de estaciones tales como:

- Estaciones de alta demanda estructural.
- Estaciones con alta variabilidad.
- Estaciones con predominancia de ciertos esquemas de pago.

8.5 5. Elección del Número de Grupos

El algoritmo K-Means requiere especificar previamente el número de clusters (k). Para determinar un valor adecuado utilizamos dos criterios cuantitativos:

- Método del codo (inercia)
- Coeficiente de silueta

Ambos permiten evaluar la calidad del agrupamiento desde distintas perspectivas: compacidad interna y separación entre grupos.

Para esto el método `evaluar_k` de la clase `ModeloClustering` construye pipelines temporales independientes para cada valor de (k), lo ajusta sobre los datos y calcula las métricas correspondientes, permitiendo que el procesamiento sea idéntico al entrenamiento final.

```
# Instanciar modelo con semilla definida
modelo_eval = ModeloClustering(n_clusters=2, seed=42)

# Evaluar k desde 2 hasta 8
inercias, silhouettes = modelo_eval.evaluar_k(X, k_max=8)

K_range_eval = range(2, 9)
```

8.5.1 5.1 Método del Codo (Inercia)

La **inercia** mide la suma de las distancias cuadradas entre cada punto y el centroide de su cluster.

A menor inercia, más compactos son los grupos.

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Gráfica del codo
axes[0].plot(K_range_eval, inercias, marker="o", color="#2c7bb6", linewidth=2, markersize=8)
axes[0].set_title("Método del Codo (Inercia)", fontsize=14, fontweight="bold")
axes[0].set_xlabel("Número de clusters (k)")
axes[0].set_ylabel("Inercia")
axes[0].grid(True, alpha=0.4)
axes[0].axvline(x=3, color="#e74c3c", linestyle="--", alpha=0.7, label="k = 3 (seleccionado)")
axes[0].legend()

# Gráfica del coeficiente de silueta
axes[1].plot(K_range_eval, silhouettes, marker="o", color="#1a9850", linewidth=2, markersize=8)
```

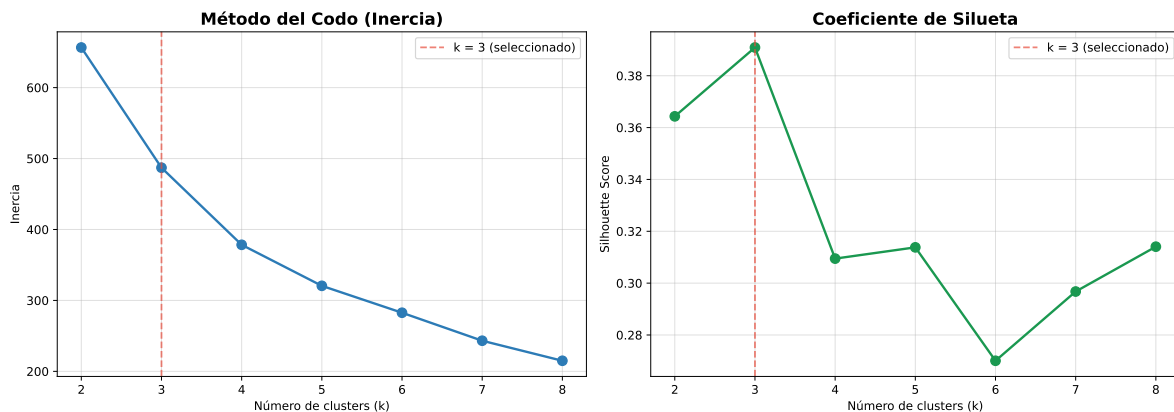


```

axes[1].set_title("Coeficiente de Silueta", fontsize=14, fontweight="bold")
axes[1].set_xlabel("Número de clusters (k)")
axes[1].set_ylabel("Silhouette Score")
axes[1].grid(True, alpha=0.4)
axes[1].axvline(x=3, color="#e74c3c", linestyle="--", alpha=0.7, label="k = 3 (seleccionado)")
axes[1].legend()

plt.tight_layout()
plt.show()

```



8.5.1.1 Interpretación

El número óptimo de clusters suele identificarse en el punto donde la curva deja de descender de forma pronunciada y comienza a aplanarse.

El **coeficiente de silueta** evalúa qué tan bien asignado está cada punto a su cluster en comparación con los demás clusters.

Su valor oscila entre -1 y 1:

- Cercano a 1 → grupos bien definidos y separados.
- Cercano a 0 → solapamiento entre clusters.
- Negativo → mala asignación.

El valor de (k) que maximiza el silhouette score indica una mejor separación entre grupos.

k	Inercia	Silhouette Score
2	656.56	0.3643
3	487.09	0.3909
4	378.41	0.3095

2	656.56	0.3643
3	487.09	0.3909
4	378.41	0.3095

5	320.58	0.3138
6	282.64	0.2701
7	243.18	0.2968
8	214.92	0.3141

8.5.2 Selección Final de (k)

Con base en los resultados obtenidos:

- El método del codo muestra una disminución pronunciada de la inercia hasta ($k = 3$), a partir del cual la mejora se vuelve marginal.
- El coeficiente de silueta alcanza su valor máximo en ($k = 3$), indicando una mejor separación entre grupos.

Por tanto, se selecciona ($k = 3$) como el número óptimo de clusters.

Esta elección equilibra adecuadamente:

- Compacidad interna de los grupos.
- Separación entre clusters.
- Interpretabilidad estructural de los perfiles de estación.

En la siguiente sección se aplicará el modelo K-Means utilizando ($k = 3$) para segmentar las estaciones del sistema.

8.6 6. Aplicación del Modelo K-Means

Con base en los criterios de inercia y coeficiente de silueta, se seleccionó ($k = 3$) como el número óptimo de clusters.

Procedemos a entrenar el modelo K-Means con este valor.

```
# Instanciar el modelo con k=3
kmeans_final = ModeloClustering(n_clusters=3, seed=42)

# Entrenar y obtener resultados
resultados = kmeans_final.entrenar(X)

print("Resultados del Entrenamiento:")
print(f"Estado          : {resultados['estado']}")
print(f"Clusters (k)     : {resultados['n_clusters']}")
print(f"Inercia           : {resultados['inercia']:.2f}")
print(f"Silhouette        : {resultados['silhouette_score']:.4f}")
```

Resultados del Entrenamiento:

Estado : entrenado
Clusters (k) : 3
Inercia : 487.09
Silhouette : 0.3909

8.6.1 6.1 Interpretación Inicial de los Clusters

Tras aplicar el algoritmo K-Means con ($k = 3$), cada estación del sistema fue asignada a uno de los tres grupos identificados.

El agrupamiento permite observar que las estaciones no se comportan de manera homogénea, sino que existen perfiles diferenciados dentro de la red.

En términos generales, los clusters representan:

- Diferencias en magnitud promedio de afluencia.
- Distintos niveles de estabilidad o volatilidad en la demanda.
- Variaciones en la composición del tipo de pago predominante.

```
# Información del modelo vía método heredado de ModeloBase  
print(kmeans_final)
```

Modelo: ModeloClustering, Estado: Entrenado, Características: ['afluencia_media', 'afluencia_std', 'proporcion_prepago']

```
# Asignar etiquetas de cluster al dataframe original  
df_cluster["cluster"] = kmeans_final.predecir(X)  
  
df_cluster[["estacion", "linea", "cluster"] + variables].head(10)
```

	estacion	linea	cluster	afluencia_media	afluencia_std	proporcion_prepago
0	BALBUENA	1	1	6826.178790	1972.007982	0.665244
1	BALDERAS	1	1	14966.389778	10277.999962	0.595050
2	BOULEVARD PUERTO AEREO	1	1	14628.169273	4195.615651	0.710051
3	CANDELARIA	1	1	16534.923182	4607.607514	0.632709
4	CHAPULTEPEC	1	2	22932.218379	16400.023590	0.680619
5	CUAUHTEMOC	1	1	8006.099122	6030.157303	0.605662
6	GOMEZ FARIAS	1	1	19097.244731	5262.724658	0.693111
7	INSURGENTES	1	2	20315.708312	16360.096648	0.722942
8	ISABEL LA CATOLICA	1	1	10843.662759	7030.782462	0.680808
9	JUANACATLAN	1	1	3543.776458	3282.701961	0.613452

8.7 7. Perfilado de los Clusters

Para interpretar los grupos identificados por K-Means, analizamos las estadísticas promedio de cada variable dentro de cada cluster.

8.7.1 7.1 Centroides en Escala Original

El método `Obtener_centroides()` de `ModeloClustering` aplica la transformación inversa del `StandardScaler` para devolver los centroides en las unidades originales, facilitando la interpretación.

```
centroides = kmeans_final.Obtener_centroides()
centroides.index.name = "Cluster"
print("Centroides por cluster (escala original):\n")
centroides.round(2)
```

Centroides por cluster (escala original):

Cluster	afluencia_media	afluencia_std	proporcion_prepago	proporcion_gratuidad	proporcion_boleto
0	14733.13	5392.06	0.83	0.12	0.05
1	12154.65	4116.98	0.68	0.16	0.16
2	37080.93	14626.58	0.74	0.10	0.15

8.7.2 7.2 Perfiles Estadísticos por Cluster

```
perfil_clusters = df_cluster.groupby("cluster")[variables].mean().round(2)
print("Perfil promedio por cluster:\n")
perfil_clusters
```

Perfil promedio por cluster:

cluster	afluencia_media	afluencia_std	proporcion_prepago	proporcion_gratuidad	proporcion_boleto
0	14733.13	5392.06	0.83	0.12	0.05

cluster	afluencia_media	afluencia_std	proporcion_prepago	proporcion_gratuidad	proporcion_boleto
1	12154.65	4116.98	0.68	0.16	0.16
2	37080.93	14626.58	0.74	0.10	0.15

8.7.3 7.3 Visualización de Perfiles

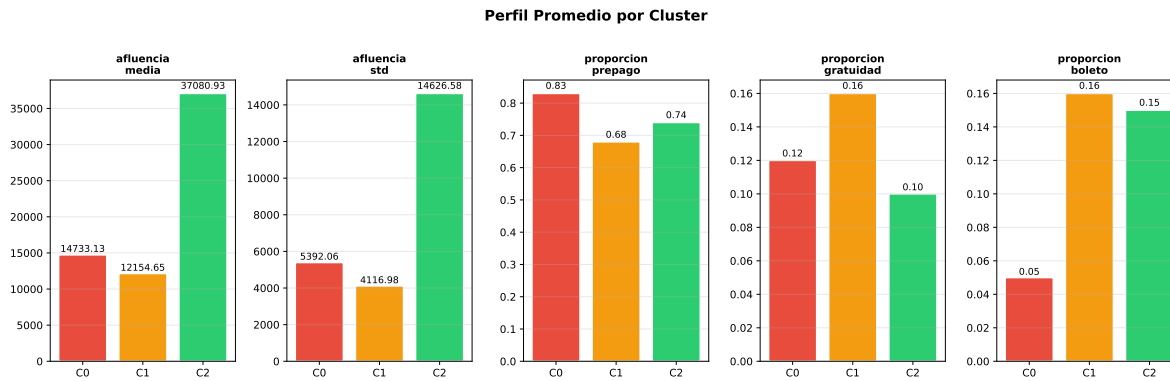
```
fig, axes = plt.subplots(1, len(variables), figsize=(16, 5))

colores_clusters = {0: "#e74c3c", 1: "#f39c12", 2: "#2ecc71"}

for i, var in enumerate(variables):
    valores = [perfil_clusters.loc[c, var] for c in range(3)]
    barras = axes[i].bar(
        [f"C{c}" for c in range(3)],
        valores,
        color=[colores_clusters[c] for c in range(3)],
        edgecolor="white",
        linewidth=1.5
    )
    axes[i].set_title(var.replace("_", "\n"), fontsize=10, fontweight="bold")
    axes[i].set_ylabel("")
    axes[i].grid(axis="y", alpha=0.3)

    for bar, val in zip(barras, valores):
        axes[i].text(
            bar.get_x() + bar.get_width() / 2,
            bar.get_height() * 1.01,
            f"{val:.2f}",
            ha="center", va="bottom", fontsize=9
        )

plt.suptitle("Perfil Promedio por Cluster", fontsize=14, fontweight="bold", y=1.02)
plt.tight_layout()
plt.show()
```



8.8 Interpretación de Perfiles

A partir de las medias observadas en cada grupo, se identifican tres perfiles estructurales diferenciados:

8.8.1 Cluster 0 — Alta Demanda Estructural

- Mayor `afluencia_media`.
- Elevada `afluencia_std` (mayor volatilidad).
- Predominio del prepago.

Corresponde a estaciones troncales o nodos estratégicos del sistema con concentración estructural alta de pasajeros.

8.8.2 Cluster 1 — Demanda Intermedia

- `Afluencia_media` moderada.
- Variabilidad media.
- Composición de pago relativamente equilibrada.

Representa estaciones con flujo constante pero sin saturación estructural extrema.

8.8.3 Cluster 2 — Baja Intensidad Estructural

- Menor `afluencia_media`.
- Variabilidad relativamente baja.
- En algunos casos mayor proporción relativa de gratuidad o boleto.

Corresponde a estaciones periféricas o con menor flujo estructural.

8.9 9. Asignación de Etiquetas Interpretativas

Para facilitar la comprensión del modelo, asignamos etiquetas descriptivas a los clusters:

```
# Usar el diccionario de nombres definido en la clase ModeloClustering
df_cluster["perfil_cluster"] = df_cluster["cluster"].map(ModeloClustering.NOMBRES_CLUSTERS)

print("Distribución de estaciones por perfil:")
print(df_cluster["perfil_cluster"].value_counts())
```

Distribución de estaciones por perfil:

```
perfil_cluster
Demanda Intermedia      130
Alta Demanda Estructural   38
Baja Intensidad          27
Name: count, dtype: int64
```

```
df_cluster[["estacion", "linea", "cluster", "perfil_cluster"]].head(10)
```

	estacion	linea	cluster	perfil_cluster
0	BALBUENA	1	1	Demanda Intermedia
1	BALDERAS	1	1	Demanda Intermedia
2	BOULEVARD PUERTO AEREO	1	1	Demanda Intermedia
3	CANDELARIA	1	1	Demanda Intermedia
4	CHAPULTEPEC	1	2	Baja Intensidad
5	CUAUHTEMOC	1	1	Demanda Intermedia
6	GOMEZ FARIAS	1	1	Demanda Intermedia
7	INSURGENTES	1	2	Baja Intensidad
8	ISABEL LA CATOLICA	1	1	Demanda Intermedia
9	JUANACATLAN	1	1	Demanda Intermedia

De esta manera, el clustering no solo segmenta numéricamente las estaciones, sino que proporciona una interpretación estructural clara y coherente con la dinámica observada en el sistema.

8.10 10. Visualización de Clusters

8.10.1 10.1 Visualización de Clusters con PCA

Para visualizar los grupos en un espacio bidimensional, aplicamos reducción de dimensionalidad mediante PCA.

```
# Obtener datos en escala estandarizada desde el pipeline interno
scaler_interno = kmeans_final.pipeline_.named_steps["scaler"]
X_scaled_final = scaler_interno.transform(X)

# Aplicar PCA sobre los datos normalizados
pca = PCA(n_components=2, random_state=42)
X_pca = pca.fit_transform(X_scaled_final)

df_cluster["pca1"] = X_pca[:, 0]
df_cluster["pca2"] = X_pca[:, 1]

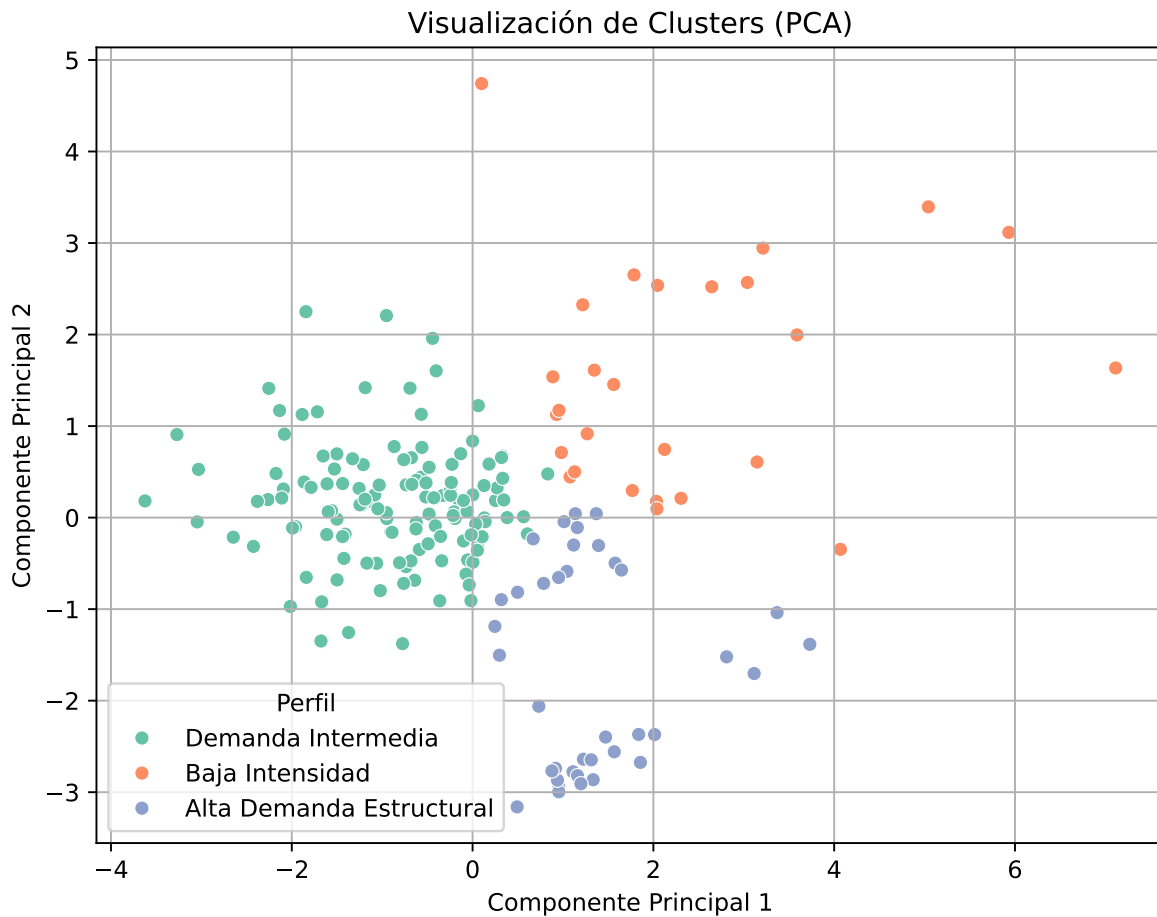
varianza_explicada = pca.explained_variance_ratio_
print(f"Varianza explicada - PC1: {varianza_explicada[0]:.1%} | PC2: {varianza_explicada[1]:.1%}")
print(f"Varianza total explicada: {sum(varianza_explicada):.1%}")
```

Varianza explicada - PC1: 52.2% | PC2: 31.6%
Varianza total explicada: 83.8%

```
plt.figure(figsize=(8,6))

sns.scatterplot(
    data=df_cluster,
    x="pca1",
    y="pca2",
    hue="perfil_cluster",
    palette="Set2"
)

plt.title("Visualización de Clusters (PCA)")
plt.xlabel("Componente Principal 1")
plt.ylabel("Componente Principal 2")
plt.legend(title="Perfil")
plt.grid(True)
plt.show()
```

La separación observada confirma que los grupos identificados presentan diferencias estructurales en el espacio de características.

8.10.2 10.2 Distribución de Estaciones por Cluster

```
df_cluster["perfil_cluster"].value_counts()
```

```
perfil_cluster
Demanda Intermedia      130
Alta Demanda Estructural   38
Baja Intensidad          27
Name: count, dtype: int64
```

Esta distribución permite identificar qué proporción de estaciones pertenece a cada perfil estructural.

8.10.3 10.3 Distribución de Clusters por Línea

Para analizar si ciertos perfiles se concentran en líneas específicas:

```
pd.crosstab(  
    df_cluster["linea"],  
    df_cluster["perfil_cluster"]  
)
```

perfil_cluster linea	Alta Demanda Estructural	Baja Intensidad	Demanda Intermedia
1	0	4	16
12	19	1	0
2	1	4	19
3	2	4	15
4	0	0	10
5	0	2	11
6	5	0	6
7	7	2	5
8	1	1	17
9	2	4	6
A	1	3	6
B	0	2	19

Esto permite identificar si determinadas líneas concentran mayor proporción de estaciones de alta demanda estructural o si presentan un comportamiento más heterogéneo.

8.11 11. Persistencia del Modelo

El modelo entrenado se guarda en disco usando el método `guardar_modelo()` heredado de `ModeloBase`. Esto permite reutilizar los centroides y el scaler sin necesidad de re-entrenar.

```
# Guardar modelo entrenado  
ruta_modelo = "../models/modelo_clustering.joblib"  
ruta_guardada = kmeans_final.guardar_modelo(ruta_modelo)  
print(f"Modelo guardado en: {ruta_guardada}")
```

Modelo guardado en: ../../models/modelo_clustering.joblib

8.12 12 Interpretación Final del Clustering

El análisis de agrupamiento confirma que la red del Metro no es homogénea, sino que presenta perfiles estructurales claramente diferenciados:

- Un grupo de estaciones troncales con alta concentración de pasajeros.
- Un grupo intermedio con demanda estable.
- Un grupo periférico de baja intensidad estructural.

Esta segmentación complementa el modelo supervisado al proporcionar una visión estructural del sistema, permitiendo identificar patrones ocultos que no se observan únicamente a partir de análisis descriptivos o clasificación diaria.

El clustering aporta una dimensión estratégica al análisis, al segmentar la red en perfiles operativos diferenciados que pueden ser utilizados para planificación, mantenimiento y gestión de saturación.

9 Arquitectura y diseño

La arquitectura y el diseño de este proyecto se basa en una arquitectura orientada a objetos (*POO*) del modulo `src`. Este módulo contiene las clases que representan los modelos de clasificación y clustering, así como una clase base para compartir funcionalidades comunes entre ellos.

Además se aplicaron patrones de diseño.

9.1 1. Arquitectura orientada a objetos

9.1.1 1.1 Visión General del Módulo `src/`

La mayor parte del código del proyecto está organizado en un paquete Python (`src/`) con siete archivos:

<code>src/</code>	
<code>__init__.py</code>	← Punto de entrada público del paquete
<code>demonstracion.py</code>	← Script de demostración para mostrar la reutilización del modelo
<code>limpieza.py</code>	← Funciones de carga y limpieza del dataset crudo
<code>modelo_base.py</code>	← Clase abstracta (contrato común)
<code>modelo_clasificacion.py</code>	← Implementación supervisada (Random Forest)
<code>modelo_clustering.py</code>	← Implementación no supervisada (K-Means)
<code>preparacion.py</code>	← Funciones de transformación y construcción de datasets derivados

9.1.2 1.2 Clases y Responsabilidades

ModeloBase (abstracta)

Define la interfaz común y funcionalidades compartidas para ambos tipos de modelos. Incluye métodos abstractos que deben ser implementados por las clases hijas, como `construir_pipeline` y `entrenar`. Además, proporciona la implementación concreta de `guardar_modelo` y `cargar_modelo` que ambas subclases heredan sin duplicación de código.

ModeloClasificacion

Implementa el modelo supervisado completo: construcción del pipeline `StandardScaler` → `RandomForestClassifier`, búsqueda de hiperparámetros con `RandomizedSearchCV` (validación

cruzada estratificada interna), evaluación multiclase con todas las métricas relevantes y análisis de importancia de variables por impureza de Gini.

Expone además constantes de clase (`NOMBRES_CLASES`, `LINEA_MAPPING`, `MES_MAPPING`) que permiten a los notebooks presentar resultados con etiquetas legibles sin duplicar la información en múltiples archivos.

ModeloClustering

Implementa el agrupamiento no supervisado. Su método `_construir_pipeline_base(n_clusters)` acepta un parámetro opcional que permite construir pipelines temporales con distintos valores de `k` durante `evaluar_k`, garantizando que el preprocesamiento sea consistente en la evaluación y en el entrenamiento final. `Obtener_centroides` aplica la transformación inversa del scaler para devolver los centroides en las unidades originales de cada variable.

9.1.3 1.3 Módulos de Preprocesamiento

Los módulos `limpieza.py` y `preparacion.py` contienen funciones específicas para la carga, limpieza y transformación de los datos, en lugar de ser clases, lo que es apropiado dado que no mantienen estado ni necesitan herencia.

limpieza.py

Contiene las funciones de carga y limpieza del dataset crudo. `cargar_datos` lee el CSV con la codificación correcta. `limpiar_dataframe` corrige los problemas de codificación de acentos en las estaciones, normaliza el campo `linea` eliminando el prefijo “Línea” y añade la variable binaria `post_boleto`. `normalizar_str` unifica mayúsculas y elimina acentos para que los joins y agrupaciones sean estables. `filtrar_cierres` elimina los registros con afluencia cero correspondientes a los cierres documentados de Líneas 1 y 12, evitando que periodos de inoperación contaminen los percentiles históricos.

preparacion.py

Contiene las transformaciones que producen los tres datasets procesados. `pivot_tipo_pago` desnormaliza los registros por tipo de pago en columnas separadas y calcula `afluencia_total`. `agregar_variable_objetivo` calcula los percentiles históricos p25, p60 y p85 por estación y los usa para construir `nivel_saturacion`, la variable target del modelo supervisado. `agregar_vars_temporales` añade `dia_semana`, `es_festivo`, `es_quincena` y `lluvia_historica`. `codificar_categoricas` aplica `LabelEncoder` a las variables categóricas. `construir_df_clustering` agrega los datos por estación calculando las proporciones de pago y estadísticos de afluencia que usa el modelo no supervisado.

9.1.4 1.4 Script de demostración

El script `demonstracion.py` muestra cómo cargar un modelo de clasificación previamente entrenado en un archivo `.joblib` y usarlo para hacer predicciones en nuevos datos. Esto

demuestra la reutilización del modelo entrenado y la persistencia del modelo.

9.2 2. Patrones de diseño aplicados

9.2.1 2.1 Patrón Strategy

El patrón Strategy te permite definir una familia de algoritmos, colocar cada uno de ellos en una clase separada y hacer sus objetos intercambiables (Refactoring Guru, n.d.). Este patrón sugiere que tomes esa clase que hace algo específico de muchas formas diferentes y extraigas todos esos algoritmos para colocarlos en clases separadas llamadas estrategias.

El patrón Strategy se refleja en la estructura de clases, donde `ModeloBase` define la interfaz abstracta común y las subclases `ModeloClasificacion` y `ModeloClustering` son las estrategias concretas que implementan estrategias específicas para clasificación y clustering respectivamente. Esto permite cambiar fácilmente entre diferentes tipos de modelos sin modificar el código que los utiliza, promoviendo la flexibilidad y la extensibilidad del proyecto.

Elegimos este patrón porque tiene una ventaja muy grande la cual es que cualquier código cliente puede operar sobre un `ModeloBase` sin necesidad de saber qué algoritmo hay debajo. Cambiar de Random Forest a Gradient Boosting, o de K-Means a DBSCAN, no requeriría modificar nada fuera de `src/`.

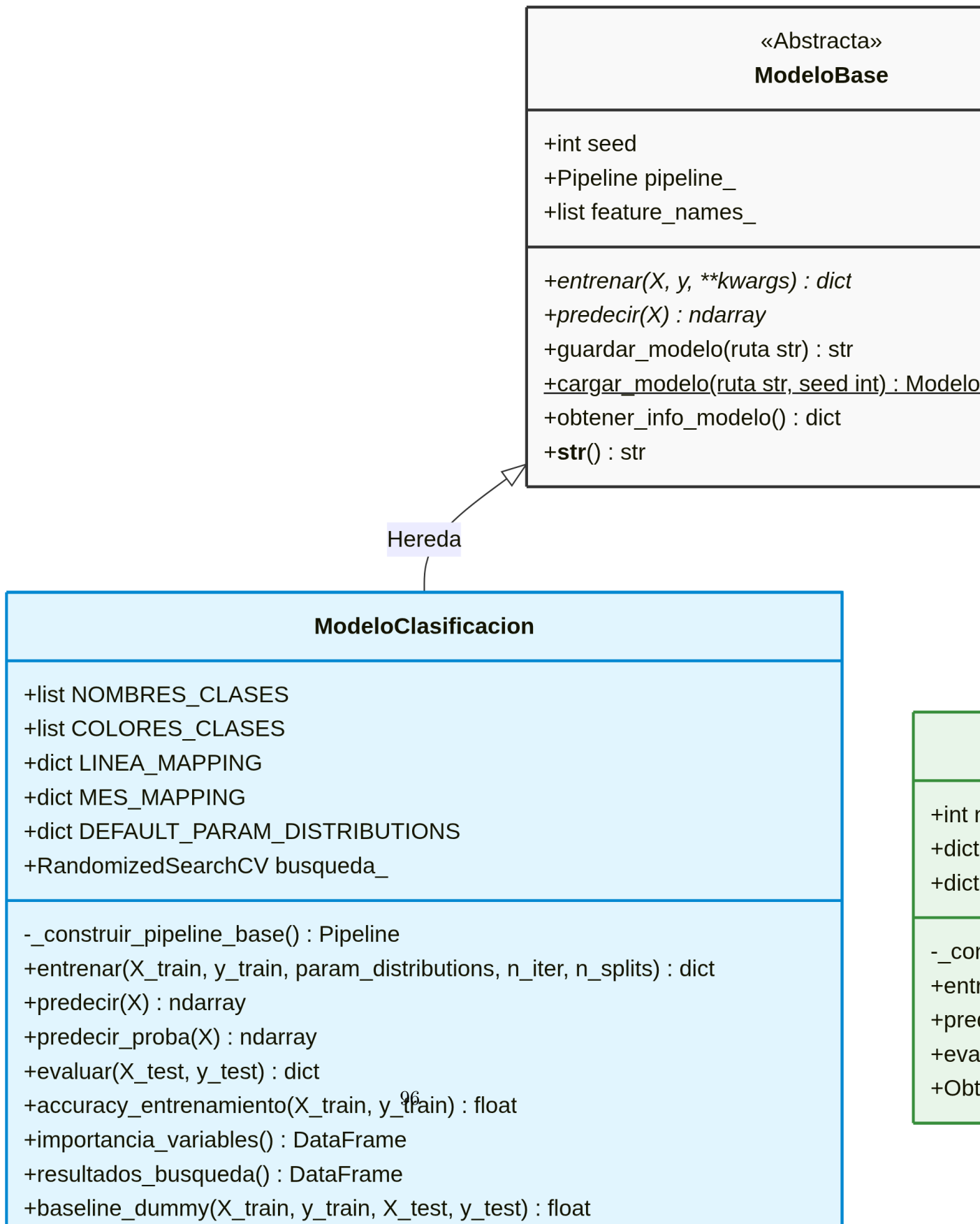
9.2.2 2.2 Patrón Pipeline

El patrón Pipeline usa una serie de etapas ordenadas para procesar los datos de entrada. Cada etapa es responsable de una parte específica del procesamiento, y el resultado de una etapa se pasa a la siguiente, lo que representa una etapa de una tubería (Iluwatar, n.d.).

Dentro de cada clase concreta, el procesamiento de datos se encapsula utilizando el patrón Pipeline de `scikit-learn`. Lo que nos garantiza que:

- El `StandardScaler` se aplica de manera consistente tanto en la evaluación de hiperparámetros como en el entrenamiento final.
- El objeto serializado con `joblib` contiene tanto el escalador ajustado como el modelo entrenado, haciendo la persistencia y la reutilización del modelo más sencilla y robusta.
- La serialización con `guardar_modelo` y la recarga con `cargar_modelo` funcionan sin problemas, ya que el pipeline completo se guarda como un solo objeto.

9.3 3. Diagrama de clases



9.4 4. Persistencia de modelos

Ambos modelos heredan los métodos `guardar_modelo` y `cargar_modelo` de `ModeloBase`, lo que garantiza una forma consistente de persistir y reutilizar los modelos entrenados. La serialización se realiza utilizando `joblib`.

```
import sys, os
sys.path.insert(0, os.path.abspath("../.."))

import pandas as pd
from src import ModeloClasificacion, ModeloClustering

# Verificar existencia de los modelos guardados
RUTA_CLASIFICACION = "../models/rf_clasificacion_metro.joblib"
RUTA_CLUSTERING    = "../models/modelo_clustering.pkl"

for ruta, nombre in [(RUTA_CLASIFICACION, "Clasificación"), (RUTA_CLUSTERING, "Clustering")]
    existe = os.path.exists(ruta)
    if existe:
        kb = os.path.getsize(ruta) / 1024
        print(f" [{nombre}] Encontrado - {ruta} ({kb:.1f} KB)")
    else:
        print(f" [{nombre}] No encontrado - ejecuta primero clasificacion.qmd y clustering.qmd")
```

```
[Clasificación] Encontrado - ../models/rf_clasificacion_metro.joblib (1714763.3 KB)
[Clustering] No encontrado - ejecuta primero clasificacion.qmd y clustering.qmd
```

10 Conclusiones

10.1 1. Hallazgos principales

Gracias a este proyecto nos podemos dar cuenta de que muchas veces es posible anticipar y caracterizar los niveles de demanda del STC Metro de la CDMX a partir exclusivamente de sus registros históricos de afluencia.

Modelo supervisado — Clasificación de saturación

El clasificador Random Forest alcanzó una *accuracy* del **69.10 %** en el conjunto de prueba y un **AUC-ROC macro de 0.8943**, partiendo únicamente de variables contextuales: identificador de línea, identificador de estación, día de la semana, mes, año y cuatro variables binarias derivadas (quincena, festivo, temporada de lluvia, era post-boleto). La mejora de **+34.40 puntos porcentuales** sobre el clasificador trivial confirma que el modelo captura patrones genuinos del sistema y no se limita a reproducir la distribución de clases.

El hallazgo más relevante del análisis de importancia de variables es que **la estación codificada explica el 38.7 % de la varianza de clasificación**, seguida del día de la semana (23.9 %) y la línea (12.9 %). Lo que quiere decir que los patrones de saturación están determinados por la ubicación y el contexto temporal, más que por factores externos como la temporada de lluvias o los festivos.

Además, el modelo de clasificación tiene la capacidad para detectar el 71% de los días de máxima saturación por lo tanto anticipar 7 de cada 10 eventos críticos. **Modelo no supervisado — Clustering de estaciones**

El clustering K-Means con $k = 3$ mostró tres perfiles:

- **Alta Demanda Estructural:** Estaciones principales con alta afluencia media y mayor volatilidad, donde predomina el pago con tarjeta.
- **Demanda Intermedia:** Estaciones con flujo constante y forma de pago mixto.
- **Baja Intensidad:** Estaciones periféricas con baja afluencia y predominio del pago en boleto o por gratuidad.

Reveló que las estrategias de operación tarifaria y control de afluencia no deben aplicarse de manera uniforme, sino adaptarse a las características estructurales de cada estación.

Este análisis no era observable a simple vista en los datos crudos.

Mientras que el modelo supervisado responde “¿qué nivel de saturación tendrá esta estación hoy?”, el clustering responde “¿qué tipo de estación es esta estructuralmente?”.

La combinación de ambos modelos permite entender no solo el nivel de saturación esperado, sino también el perfil de cada estación, lo que puede ser útil para la planificación operativa y la asignación de recursos.

10.2 2. Limitaciones del análisis

A pesar de la robustez de los modelos, el análisis está acotado por la naturaleza de los datos disponibles en el portal abierto:

Ausencia de capacidad instalada

El dataset no incluye información sobre la capacidad máxima por línea o por estación. Esto limita la interpretación de los niveles de saturación, ya que no se puede cuantificar el grado de sobrecarga en términos absolutos, sino solo en relación a su propia historia.

Eventos no documentados

Cierres temporales por mantenimiento, accidentes, eventos masivos en zonas aledañas o cortes de servicio no programados afectan la afluencia registrada pero no están identificados en el dataset. Los dos cierres documentados (Línea 12 y Línea 1) fueron filtrados explícitamente, pero episodios menores o no documentados pueden introducir ruido en los percentiles históricos y por tanto en la variable objetivo.

Fronteras de Decisión Difusas

La matriz de confusión del modelo supervisado muestra que los errores ocurren casi exclusivamente entre clases adyacentes (ej. confundir un día “Alto” con “Medio”).

10.3 3. Trabajo Futuro

El paso más valioso para mejorar el modelo supervisado sería incorporar datos adicionales que permitan cuantificar la capacidad instalada y los eventos disruptivos. Esto podría incluir:

- Datos de capacidad por estación y por línea, para transformar la variable objetivo en una medida de saturación relativa a la capacidad.
- Registros de eventos disruptivos, para filtrar o etiquetar los días afectados y evitar que contaminen los patrones históricos.

También se podrían incorporar datos meteorológicos más detallados, como temperatura o eventos climáticos extremos, para evaluar su impacto en la afluencia y la saturación.

Referencias

- Comisión Nacional de los Salarios Mínimos. 2017. *Nuevo Salario Mínimo General \$88.36 Pesos Diarios*. Gobierno de México. <https://www.gob.mx/conasami/articulos/nuevo-salario-minimo-general-88-36-pesos-diarios?idiom=es>.
- Constantino, Paul. 2024. *¡Adiós Boleto Del Metro CDMX! La Historia Del Cartoncito Magnético Llegó a Su Fin*. El economista. <https://www.eleconomista.com.mx/estados/Adios-boleto-del-Metro-CDMX-La-historia-del-cartoncito-magnetico-llego-a-su-fin-20240420-0017.html>.
- El Financiero. 2025. *¡Al Fin! Línea 1 Del Metro CDMX Reabre En Su Totalidad: Así Fue Su Remodelación Que Tomó Más de 3 Años*. <https://www.elfinanciero.com.mx/cdmx/2025/11/16/metro-cdmx-reabre-tramo-juanacatlan-observatorio-asi-fueron-las-remodelaciones-de-la-linea-1-que-tomaron-mas-de-3-anos/>.
- Iluwatar. n.d. *Patrón de Diseño Pipeline*. Java Design Patterns. <https://java-design-patterns.com/es/patterns/pipeline/#proposito>.
- Once Noticias. 2026. *Metro de La CDMX Rebasa Los 100 Millones de Personas Usuarías En Enero*. Once Noticias Digital. <https://oncenoticias.digital/valle-de-mexico/metro-de-cdmx-rebasa-los-100-millones-de-personas-usuarias-en-enero/554910/>.
- Refactoring Guru. n.d. *Patrón de Diseño Strategy*. Refactoring Guru. <https://refactoring.guru/es/design-patterns/strategy>.
- Sarabia, Dalila. 2024. *Hoy Reabre La Línea 12 Del Metro; Dos Años y Siete Meses Sin Culpables Por Colapso y de La Muerte de 26 Personas*. Animal Político. <https://grupoanimal.mx/estados/linea-12-metro-del-metro-hoy-reabre>.
- Sistema de Transporte Colectivo Metro. 2013. *ESTUDIO CUANTITATIVO SOBRE EL SERVICIO DEL STCM*. Presentación de resultados. Gobierno de la Ciudad de México. <https://metro.cdmx.gob.mx/storage/app/media/Metro%20Acerca%20de/preguntas%20frecuentes/encservicios1.pdf>.
- Sistema de Transporte Colectivo Metro. 2018. *ENCUESTA SOBRE LA PERCEPCIÓN DE LA CALIDAD DEL SERVICIO CON PERSPECTIVA DE GÉNERO, DERECHOS HUMANOS*

Y NO DISCRIMINACIÓN. Presentación de resultados. Gobierno de la Ciudad de México.
https://metro.cdmx.gob.mx/storage/app/media/inicio/encuesta_dh_nov18.pdf.

Sistema de Transporte Colectivo Metro. n.d. *Acerca de*. Gobierno de la Ciudad de México.
<https://www.metro.cdmx.gob.mx/organismo/acerca-de>.